

DOCUMENTATION PACKAGE · V0.1 DRAFT FOR REVIEW

Finance Core Blueprint

Architecture, requirements, data model, technical design, and use cases for Warraq's new General Ledger / Accounts Payable / Accounts Receivable core — replacing today's document-derived bookkeeping with a real double-entry ledger that Sales, Inventory, Procurement, and HR/Timekeeping all settle into.

Status

For review — no code changes made

Scope

GL · AP · AR · Branch/Dept costing · Tax posting · Payroll-ready dimensions

Out of scope (this phase)

Multi-currency · Intercompany · Payroll calculation engine

Companion

Full architecture discussion & sample Chart of Accounts (this session)

DOCUMENT 1

Architecture & Integration Flow

How Sales, Purchase, Inventory, HR/Payroll, and Bank/Cash all converge on one posting substrate — and what each one hands the ledger when it fires.

CORE IDEA

There is exactly one place money gets recorded: the Journal Entry engine. Every other module is a **source** — it decides *when* and *how much*, then hands the ledger a balanced set of debit/credit lines. AP, AR, Trial Balance, P&L, and Balance Sheet are not separate systems; they are queries over the same rows.

1.1 The one cost-distribution primitive

Branch overhead allocation, manufacturing overhead absorption, project costing, landed-cost allocation on imports, and shared fixed-asset depreciation are not five features — they are five configurations of one primitive: **trace what's certain, pool what's shared, distribute the pool by one formula.**

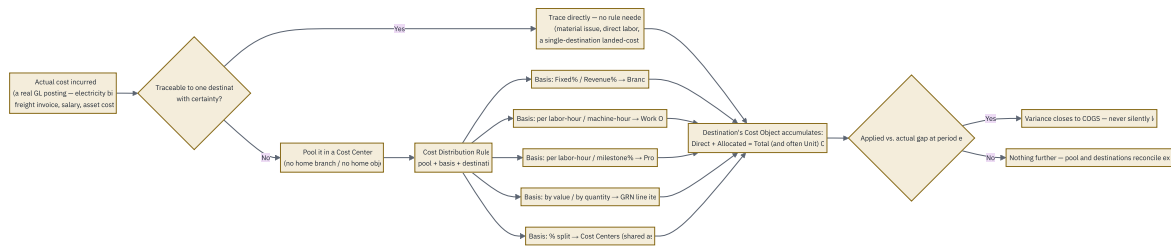


Fig. 1a — One engine, five configurations. Same preview-then-post run, same idempotency-per-period guarantee, same true-up discipline, every time.

A **Cost Object** is anything on the receiving end that sums Direct + Allocated into a Total: a Branch (→ P&L), a Work Order (→ unit manufacturing cost), a Project (→ actual-vs-budget), or an inventory item at receipt (→ true landed cost). A Fixed Asset is the mirror image, not a sixth instance — one known cost that itself *becomes* a pool (depreciation), distributed onward, rather than a thing that accumulates cost from multiple streams.

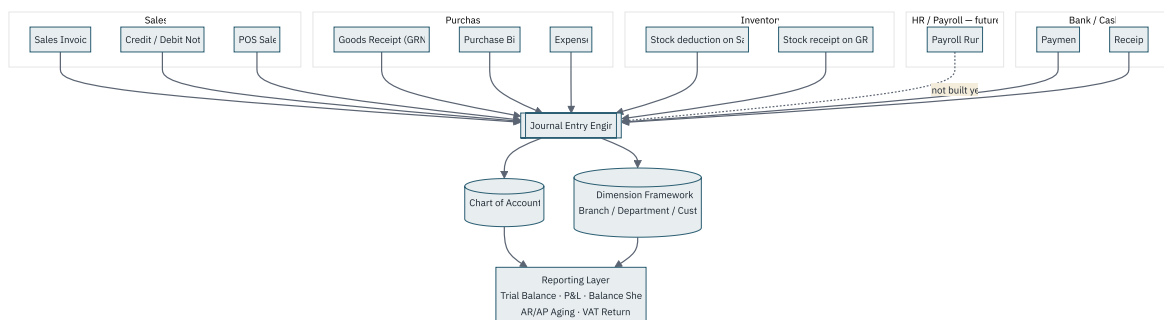


Fig. 1 — Every source module posts into one engine; reporting reads back out of it.

End-to-end: a sale, paid in two installments

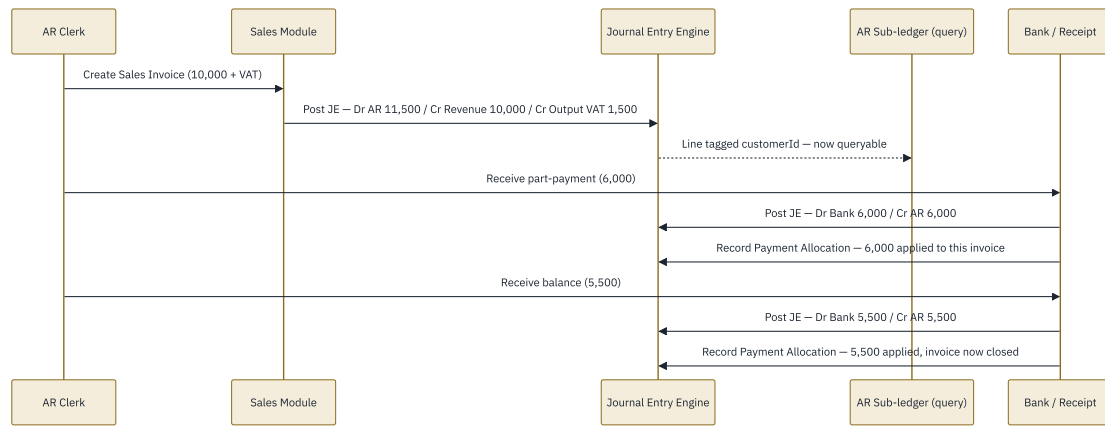


Fig. 2 — AR balance is always the sum of unallocated AR lines, never a separately-tracked field.

Source → posting map

SOURCE MODULE	EVENT	SOURCETYPE	ACCOUNTS TOUCHED
Sales	Sales Invoice posted	SalesInvoice	Dr AR · Cr Revenue · Cr Output VAT Payable
Sales	Credit / Debit Note	CreditNote / DebitNote	Reverses the invoice's proportional lines
Sales	Sale reduces stock	COGS	Dr COGS · Cr Inventory Asset, at weighted-avg cost
Purchase	Goods Receipt (GRN)	InventoryReceipt	Dr Inventory Asset · Cr GRNI (clearing)
Purchase	Purchase Bill matched to GRN	PurchaseBill	Dr GRNI · Dr Input VAT Receivable · Cr AP
Purchase	Expense (no PO)	PurchaseBill	Dr Expense · Dr Input VAT · Cr AP — same path as a bill
Bank/Cash	Payment issued	Payment	Dr AP · Cr Bank, + Payment Allocation row(s)
Bank/Cash	Receipt collected	Receipt	Dr Bank · Cr AR, + Payment Allocation row(s)
Cost/Branch	Month-end allocation run	Allocation	Cr pool's Allocated-Overhead acct · Dr each destination branch, same account
Tax	Quarter filed	TaxSettlement	Dr Output VAT · Cr Input VAT · net to VAT Payable to ZATCA
Period	Fiscal month closed	PeriodClose	Zero every Revenue/Expense acct · net to Retained Earnings
HR/Payroll future	Payroll run	Payroll	Dr Payroll Expense accts (per element, per cost center) · Cr Salaries Payable / GOSI / EOSB

Detailed Business Requirements Document

What the system must do, in plain terms, grouped by area — the reference both engineering and finance sign off against.

2.1 Purpose & background

Warraq's Accounts/Finance module today computes Trial Balance, P&L, and Balance Sheet by re-deriving a fixed set of buckets from source documents on every render — there is no chart of accounts, no double-entry ledger, and Purchase Bills post no liability until cash actually moves. Fiscal-month close trusts a browser-computed P&L figure with no server-side verification. This BRD defines the replacement: a real general ledger that Sales, Inventory, Procurement, and the upcoming HR/Timekeeping module all post into, meeting the standard of accuracy and auditability a tier-1 ERP is held to.

2.2 Objectives

Accuracy

- Every financial statement traces to an immutable, balanced ledger — never a re-derived approximation
- AR/AP sub-ledgers cannot drift from the GL — they are the same rows, not parallel calculations
- Period close is server-verified, not browser-trusted

Integration readiness

- Sales, Purchase, and Inventory post through one substrate today
- HR/Timekeeping payroll can post through the same substrate on day one of that module, with zero ledger redesign
- Branch and Department P&L are a query, not a manual reconciliation exercise

2.3 Scope

IN SCOPE

OUT OF SCOPE (THIS PHASE)

Chart of Accounts, Journal Entry engine, AR/AP sub-ledger & payment allocation, Inventory→COGS integration, dynamic Branch/Department dimension framework & allocation engine, VAT settlement posting, server-verified period close, payroll-ready dimension tagging, rounding rules (tax-line and cash-tender)

Multi-currency & FX revaluation, intercompany accounting, consolidation, the actual Payroll calculation engine (only its GL posting contract is defined here), collections/dunning workflow, payment proposals, positive pay

2.4 Stakeholders / actors

Accountant/Bookkeeper, AP Clerk, AR Clerk, Branch Manager, Finance Controller, Super Admin, Auditor (read-only), and the System itself as an automated poster — fully detailed in the Use Case documents below.

2.5 Functional requirements

Chart of Accounts

FR-COA-01 The system shall maintain a hierarchical, company-scoped Chart of Accounts with account code, name, type (Asset/Liability/Equity/Revenue/Expense), sub-type, and normal balance.

FR-COA-02 The system shall flag accounts as control accounts (AR, AP, Inventory, VAT) and reject any manual journal line posted to a control account.

FR-COA-03 The system shall seed a standard starter Chart of Accounts automatically when a new company is provisioned.

FR-COA-04 The system shall prevent deletion of an account with any posted journal line; deactivation is the only retirement path.

FR-COA-05 The system shall show, for any account, every other configuration record that references it (product mappings, bank linkage, tax linkage) before allowing deactivation.

Journal Entries

FR-JE-01 The system shall reject any journal entry whose total debits do not equal its total credits, evaluated server-side inside the same transaction as the insert.

FR-JE-02 The system shall never allow a posted journal entry to be updated or deleted; corrections are a reversing entry plus a new correct entry.

FR-JE-03 Every journal entry shall record its source module and source document, and every line shall carry the dimension tags relevant to its account (Branch, Department, Customer, Vendor, or Tax Code as applicable).

FR-JE-04 The system shall assign a unique, sequential entry number per company via the existing document-numbering engine.

Approval Cycle (Journal Entries, Vendor Bills, Payments, Credit Notes)

FR-APR-01 The system shall support configurable approval rules per document type (Journal Entry, Vendor Bill, Vendor Payment, Credit Note) keyed on amount thresholds, optionally requiring multiple sequential levels above higher thresholds.

FR-APR-02 Document matching an active rule shall enter a Pending Approval state and shall not post — and for a Vendor Payment, shall not release funds — until every required level has approved.

FR-APR-03 The system shall prevent a user from approving their own submission at any level, enforced server-side, not merely hidden in the UI.

FR-APR-04 Every approval or rejection shall be recorded with actor, timestamp, level, and optional remarks, visible in the document's own history.

FR-APR-05 Rejected document shall return to Draft with the rejection remarks visible to the submitter; nothing rejected ever reaches the ledger.

FR-APR-06 Document with no matching approval rule shall post immediately through the existing direct-posting path — approval is rule-driven overhead on high-value or sensitive transactions, not a blanket requirement on every entry.

Accounts Receivable

FR-AR-01 Posted Sales Invoice, Credit Note, or Debit Note shall automatically generate a balanced journal entry against the AR control account, tagged with the customer.

FR-AR-02 Customer receipt shall be applicable across one or more open invoices, or held unapplied on account, via a payment allocation record.

FR-AR-03 Aging and customer statements shall be computed as a live query over posted journal lines against the AR control account — never a separately maintained balance.

Accounts Payable

FR-AP-01 Goods Receipt shall recognize inventory value against a clearing (GRNI) account at the moment goods are received, independent of when the vendor bill arrives.

FR-AP-02 matched Purchase Bill shall clear the GRNI balance and recognize the AP liability at that moment — not when cash is later paid.

FR-AP-03 expenses without a PO shall post through the same AP control account and the same vendor sub-ledger as Purchase Bills — one vendor position, not two disconnected flows.

FR-AP-04 vendor payment shall be applicable across one or more open bills, mirroring AR settlement.

Landed Cost / Import Costing

FR-LC-01 an import GRN shall support any number of cost lines beyond the item price — zero, two, five, or however many a given shipment actually carries — drawn from a company-configured list of cost types (Freight, Customs Duty, Insurance, Clearing/Commission Agent fee, Bank/LC charges, or anything else the company defines), each independently payable to its own vendor or government authority, not forced onto the goods vendor's bill. The line count is never fixed or predetermined — one shipment might need two lines, another five, decided freely at entry time.

FR-LC-02 each cost type shall carry a configurable default GL treatment (Capitalize into Inventory, or Expense Directly), overridable per transaction, so freight and duty capitalize into stock value while a bank charge, say, can expense immediately per company policy.

FR-LC-03 capitalized cost line shall be allocated across the GRN's item lines by a chosen basis (By Value, By Quantity, or Manual), so each item's landed unit cost reflects its true fully-loaded cost, not just its vendor invoice price.

FR-LC-04 each cost line shall accrue at GRN time and clear into a real Accounts Payable liability to its specific vendor only once that vendor's own bill is matched — mirroring the existing GRNI mechanism, applied per cost component.

FR-LC-05 the weighted-average cost used for COGS shall be computed from the item's fully landed cost, not its invoice price alone, once any landed-cost sheet has been posted against it.

Manufacturing / Work-in-Progress Costing forward design — no production module yet

FR-WIP-01 the inventory model shall distinguish Raw Material, Work-in-Progress, and Finished Goods as separate inventory asset accounts, each independently reconcilable, rather than one undifferentiated stock value.

FR-WIP-02 Work Order shall accumulate cost from exactly three sources: material issued from Raw Material (at its own current landed cost), **direct** labor and machine-operator time traceable to that specific Work Order, and a share of **indirect** manufacturing overhead — each debits WIP as it's incurred, but the two labor-and-beyond components are fundamentally different and must not be conflated.

FR-WIP-03 Direct cost — an operator running a machine on this job, a worker assembling this batch — shall be logged per Work Order, per employee, per hours, and traced 1:1 into that Work Order's cost. This is the same shape a future Timekeeping module would capture, so a clocked production-floor entry can feed both payroll and Work Order costing from one source, not two.

FR-WIP-04 Indirect cost — a supervisor's salary, factory electricity and gas, factory rent, production-equipment depreciation, maintenance staff — cannot be traced to one specific Work Order and shall instead post to their own actual expense accounts as incurred (a real electricity bill is still a real AP transaction, unchanged from how any other bill posts), then be pooled as Manufacturing Overhead and **applied** to Work Orders via a configured absorption rate (per direct labor hour, per machine hour, or per unit) — never by trying to split one utility bill across jobs by hand.

FR-WIP-05 Applied overhead shall post to a clearing account distinct from the actual overhead expense accounts (Factory Utilities, Indirect Labor, Factory Rent, Production Equipment Depreciation) it's meant to absorb; at period close, the difference between what was applied to Work Orders and what was actually spent in the pool shall true up to Cost of Goods Sold, so no cost is ever silently lost or double-counted between "what jobs were charged" and "what the factory actually spent."

FR-WIP-06 Completing a Work Order shall transfer its full accumulated cost — material, traced direct labor, and applied overhead together — from WIP to Finished Goods, computing a per-unit cost that becomes that batch's cost basis. From that point forward a manufactured item is indistinguishable from a purchased one for COGS-at-sale purposes.

FR-WIP-07 Scrap or wastage identified during production shall post from WIP to the existing Inventory Write-off/Shrinkage account, not silently vanish from the Work Order's accumulated cost.

FR-WIP-08 Project (service, installation, or custom-order work rather than physical manufacturing) shall accumulate cost through the identical mechanism as a Work Order — traced material, traced labor, applied overhead via the same Cost Distribution Rule engine — reporting actual-vs-budgeted cost rather than a per-unit cost, since a project is not typically divided into identical units.

See [§1.1](#) — Work Order and Project overhead absorption, branch overhead reclass, landed-cost allocation, and shared fixed-asset depreciation are all the same Cost Distribution primitive, not four separate mechanisms.

Bank Reconciliation

FR-BANK-01 system shall support a reconciliation session per bank account per statement period, capturing the statement ending balance and date.

FR-BANK-02 system shall list every posted, uncleared journal line against that bank account's linked GL account, and allow marking each as cleared against imported or manually entered statement lines.

FR-BANK-03 system shall compute and display the reconciling formula live — Book Balance + Outstanding Deposits – Outstanding Payments = Statement Balance — and block completion while it doesn't balance.

FR-BANK-04 the discrepancy (bank fee, interest, timing error) shall be resolved by posting a journal entry during the session, never by force-completing an unbalanced reconciliation.

FR-BANK-05 completed reconciliation shall lock its cleared lines against further edits; reopening one shall require elevated permission and be recorded in the audit trail.

Branch / Department Costing

FR-DIM-01 the system shall support a company-configurable set of dimension types (starting with Branch and Department, seeded automatically), each independently combinable on any journal line.

FR-DIM-02 department/Cost Center records shall support an optional home Branch; a Cost Center with no home Branch is eligible for allocation.

FR-DIM-03 the system shall support a configurable monthly allocation run that redistributes a shared Cost Center's postings across destination branches by a fixed-percentage or revenue-percentage basis, posted to a dedicated Allocated-Overhead account, net zero at company level.

FR-DIM-04 Branch P&L shall present Direct Expenses and Allocated Overhead as distinct subtotals (Contribution Margin, then fully-loaded Branch Net Profit).

Tax

FR-TAX-01 every taxed transaction shall post its tax amount to a dedicated Output/Input VAT account, tagged with its tax code, never blended into revenue or expense.

FR-TAX-02 ending a quarter shall post a real settlement journal entry netting Output VAT against Input VAT into VAT Payable to ZATCA, replacing today's re-derived snapshot.

Fixed Assets & Depreciation

FR-FA-01 the system shall maintain Asset Classes (e.g. Furniture & Fixtures, Vehicles, IT Equipment) each carrying a default useful life, depreciation method, and default Cost/Accumulated-Depreciation/Depreciation-Expense accounts, inherited by an asset on creation and overridable per asset.

- FR-FA-02** An asset acquired mid-construction/fit-out shall be recorded against Capital Work-in-Progress and shall not depreciate until explicitly moved to In-Service status, at which point its cost transfers from CWIP to its real Fixed Asset Cost account.
- FR-FA-03** The system shall support Straight-Line, Declining Balance, and Units-of-Production depreciation methods, computed per asset from its cost, salvage value, useful life or rate, and (for units-of-production) usage.
- FR-FA-04** A depreciation run for a period shall compute each in-service asset's amount, write it to a per-asset depreciation schedule, and post one journal entry — debiting Depreciation Expense (by account and dimension), crediting Accumulated Depreciation — grouped, not one GL line per asset.
- FR-FA-05** A depreciation run shall be previewable before posting and shall never run twice for the same asset in the same period.
- FR-FA-06** Disposing an asset shall post a single entry clearing its Cost and Accumulated Depreciation, recording any sale proceeds, and posting the resulting Gain or Loss on Disposal — never left as a dangling balance.
- FR-FA-07** Retiring an asset (taken out of service with no sale) shall follow the same clearing mechanics as Disposal with proceeds fixed at zero — the full remaining net book value posts to Loss on Disposal — recorded as a distinct status (Retired vs. Disposed) for reporting, not a separate posting mechanism.
- FR-FA-08** Transferring an asset between branches, cost centers, or custodians shall update its dimension tag and custodian from the transfer date forward, with no retroactive change to depreciation already posted, and shall record a from/to transfer history entry.
- FR-FA-09** An asset used across more than one cost center shall support a percentage cost-allocation split, so its depreciation run posts proportionally across all listed cost centers instead of a single dimension.
- FR-FA-10** Every transfer and every disposal/retirement shall produce a printable form (Asset Transfer Note, Asset Disposal/Retirement Note) carrying the asset's identity, cost, accumulated depreciation and NBV at the event date, the from/to detail, requester, and a signature section — following the same print-template mechanism as every other movement document in the app (GRN, Delivery Note, Stock Transfer Note).

Monthly Period Lock

- FR-CLOSE-01** Closing a fiscal month shall compute that month's trial balance from actually-posted journal lines, server-side, re-verify debit=credit, and **lock** the period against further postings — it shall not zero any Revenue or Expense account. Revenue and Expense

accumulate continuously through the fiscal year; only Fiscal Year-End Close (below) zeros them.

FR-CLOSE-02 sed month's verified trial balance shall be cached for reporting, but P&L for any date range shall always be computable as a live sum over posted lines — the cache is a performance aid, never the only source.

Fiscal Year-End Close

FR-CLOSE-03 al year shall not be closeable until every month within it is Closed, every bank reconciliation in it is Completed, every in-service asset's depreciation has run for every month in it, every VAT quarter in it is Filed, every Cost Distribution run in it has executed, and no document dated within it remains Draft or Pending Approval — presented as a single readiness checklist, not left to memory.

FR-CLOSE-04 End Close shall compute the full year's net income server-side from posted journal lines and post exactly one closing entry: zero every Revenue and Expense account's full-year balance into Retained Earnings.

FR-CLOSE-05 nce Sheet accounts require no carry-forward posting of any kind — their balance is the continuous running total of every line ever posted. Only Revenue/Expense (zeroed) and Retained Earnings (incremented) are touched by Year-End Close.

FR-CLOSE-06 End Close shall automatically open the new fiscal year's first month, so operations continue without a manual setup step.

FR-CLOSE-07 sed fiscal year shall support a restricted Adjustment Period — a final window, gated to Controller/Auditor roles only, for genuine post-close correcting entries — rather than ever reopening a closed year's operating months.

FR-CLOSE-08 truly closed, a fiscal year is never reopened; a correction discovered later posts as a disclosed prior-period adjustment in the current year, not a rewrite of closed history.

Rounding

FR-RND-01 oice line tax amounts shall be computed at higher internal precision and rounded to 2 decimal places per line; the invoice total shall be the sum of already-rounded lines.

FR-RND-02 sh-tender rounding, where enabled per company, shall never alter the invoice's legal total; the rounding delta shall post to a dedicated Cash Rounding Adjustment account.

2.6 Non-functional requirements

CATEGORY	REQUIREMENT
Integrity	Balance invariant ($\Sigma\text{debit}=\Sigma\text{credit}$) enforced server-side on every insert, not merely validated client-side
Auditability	Every posted entry is immutable; every correction is traceable to its reversal
Multi-tenancy	Every new table carries companyId; reuses the existing isolation pattern, verified by the project's own isolation-guard tooling
Performance	Dimension tagging resolves through a single surrogate key per line (not a multi-way join per report query)
Compatibility	Rollout is additive at every step; no existing route's behavior changes until explicitly cut over
Compliance	Posted tax figures reconcile exactly to whatever the ZATCA e-invoice XML already reports for the same transaction

2.7 Business rules

- A control account can never receive a manual journal line — only its owning subledger may post to it.
- Nothing is ever deleted once posted; only reversed.
- A quarter marked Filed is permanently locked against new or edited transactional documents, no role can override it.
- An allocation entry always uses a dedicated Allocated-Overhead account, never the original natural expense account.
- Department stays constant through an allocation entry; only Branch changes.
- A document awaiting approval is invisible to every report that reads the ledger — nothing pending is ever mistaken for posted.
- No one approves their own submission, at any approval level, for any document type.
- An asset under construction (CWIP) never depreciates; the clock starts only from its In-Service date.
- A depreciation run is idempotent per asset per period — re-running it changes nothing already posted.
- Monthly close locks a period; it never zeros an account. Only Fiscal Year-End Close does, exactly once per year.
- A closed fiscal year is never reopened — a later correction is a disclosed adjustment in the current year, not a rewrite of history.

2.8 Acceptance criteria

- Every journal entry in the system balances to zero, with no exceptions, verifiable by a single aggregate query.
- AR control account balance equals the sum of open AR sub-ledger lines, exactly, at any point in time.
- A real fiscal month's GL-derived Trial Balance matches the legacy computed Trial Balance during the dual-posting verification window.
- Branch P&L subtotals (Direct vs. Allocated) sum to the correct company-wide total for every account.

2.9 Reporting requirements

Grouped by module. The single most important item in this set is last — **Trial Balance and P&L stop being two hardcoded reports and become two seeded instances of one configurable Report Definition engine**, so any GL-level report can be built the same way without touching code.

General Ledger reports

FR-RPT-GL-01 System shall provide a General Ledger Detail report: every posted line for a chosen account (or account range), date range, and dimension filter (branch/department/custom), with a running balance column.

FR-RPT-GL-02 System shall provide a Journal Listing report: every posted entry in a period, filterable by source type (Manual/SalesInvoice/PurchaseBill/Payment/...), with drill-down to its lines.

FR-RPT-GL-03 System shall provide an Audit Trail report of reversed entries, pairing each reversal with the entry it reverses and the user/timestamp of both.

FR-RPT-GL-04 summary figure in any GL report shall be clickable through to the General Ledger Detail lines that produced it — no report may present a number with no drill-down path.

FR-RPT-GL-05 Asset Register — every asset with cost, accumulated depreciation, and net book value as of any date, filterable by class and branch, reconciling to the Cost and Accumulated Depreciation control accounts.

Accounts Payable reports

FR-RPT-AP-01 Aging — outstanding vendor balances bucketed by days overdue (configurable bucket boundaries), filterable by vendor and branch.

FR-RPT-AP-02 r Statement — one vendor's bills, payments, and running balance over a period, print-ready.

FR-RPT-AP-03 Reconciliation — open Goods-Received-Not-Invoiced balances, flagging anything unmatched beyond a configurable age threshold.

FR-RPT-AP-04 ase VAT Register — every input-tax line in a period, grouped by tax code, reconciling to the Input VAT Receivable control account.

Accounts Receivable reports

FR-RPT-AR-01 ing — outstanding customer balances bucketed by days overdue, filterable by customer and branch.

FR-RPT-AR-02 mer Statement — one customer's invoices, receipts, and running balance over a period, print-ready.

FR-RPT-AR-03 VAT Register — every output-tax line in a period, grouped by tax code, reconciling to the Output VAT Payable control account.

FR-RPT-AR-04 olid Receipts — customer receipts held on-account, not yet allocated to an invoice, so nothing sits invisibly unmatched.

Additional standard statements & reports

FR-RPT-STMT-01 ow Statement (indirect method) — net income adjusted for non-cash items (depreciation, applied-overhead variances) and working-capital movement (AR/AP/Inventory change), reconciling to the actual change in Cash and Bank balances for the period.

FR-RPT-STMT-02 nt of Changes in Equity — opening equity, capital contributions (from the existing investor records), net income for the period, any distributions, to closing equity — the report-level view of what Fiscal Year-End Close posts.

FR-RPT-STMT-03 sition Summary — every bank and cash account's current balance in one view, for daily treasury visibility, distinct from the per-account Bank Ledger.

FR-RPT-STMT-04 ation Schedule — projected depreciation by asset for future periods, distinct from the Fixed Asset Register's as-of-date snapshot; used for forward planning.

FR-RPT-STMT-05 uation — every open Work Order/Project's accumulated cost as of a date, reconciling to the WIP control account balance.

FR-RPT-STMT-06 Goods Manufactured Statement — $\text{Opening WIP} + \text{Material Consumed} + \text{Direct Labor} + \text{Overhead Applied} - \text{Closing WIP} = \text{COGM}$, the standard manufacturing statement bridging production cost to Finished Goods.

FR-RPT-STMT-07 :turing Overhead Variance — applied vs. actual overhead by Cost Distribution Rule and period, surfacing exactly what FR-WIP-05's true-up closes to COGS.

FR-RPT-STMT-08 Profitability — actual cost vs. budgeted cost per project, margin, status.

FR-RPT-STMT-09 .l History — every submitted/approved/rejected request over a period, by document type, actor, and level — the standalone report view of what the approval cycle already records per-document.

Standard report catalog — full index

CATEGORY	REPORT	STATUS
Primary statements	Trial Balance	Existing – GL-driven
Primary statements	Profit & Loss / Income Statement	Existing – GL-driven
Primary statements	Balance Sheet	Existing – GL-driven
Primary statements	Cash Flow Statement	New
Primary statements	Statement of Changes in Equity	New
Sub-ledger & aging	AR Aging, AP Aging	Existing – GL-driven
Sub-ledger & aging	Outstanding Summary (AR+AP combined)	Existing – GL-driven
Sub-ledger & aging	Customer Statement, Vendor Statement	Existing – GL-driven
Sub-ledger & aging	Unapplied Receipts	New
Tax & compliance	Sales VAT Register, Purchase VAT Register, VAT Return Summary	Existing – GL-driven
Tax & compliance	GRNI Reconciliation	New
Cash & treasury	Bank Ledger / Statement	Existing
Cash & treasury	Cash Position Summary	New
Fixed assets	Fixed Asset Register	New
Fixed assets	Depreciation Schedule	New
Manufacturing & project	WIP Valuation, Cost of Goods Manufactured, Overhead Variance,	New

CATEGORY	REPORT	STATUS
	Project Profitability	
Management & comparative	Branch P&L Comparison, Comparative P&L (period/YoY)	Existing – via configurable columns
Management & comparative	Investor Profit Share	Existing – GL-driven
Governance & audit	General Ledger Detail, Journal Listing, Audit Trail (reversals)	Existing
Governance & audit	Approval History	New
Deferred	Budget vs. Actual (GL-level) — no company-wide budget entity in this design; Work Order/Project actual-vs-budget already covered	Deferred

Existing — GL-driven means this report already exists in Warraq today and carries forward under this design, now computed from real posted journal lines instead of re-derived from source documents. Every report in this catalog, new or carried-forward, is either a seeded Report Definition (\$FR-RPT-CFG) or a purpose-built query over the same journal-entry data — none of them are a second, parallel calculation.

FR-RPT-CFG — THE CONFIGURABLE TRIAL BALANCE / P&L REQUIREMENT

Both Trial Balance and P&L shall be produced by one generic Report Definition engine, not two hardcoded functions. A Report Definition is a named, saved combination of:

Row Definitions — each row selects its accounts by Range (e.g. 6000–6999), Parent Rollup (an account plus every descendant under it), or an explicit account list, and — regardless of which base selection is used — carries its own **Include** and **Exclude** account lists applied on top, so a row like "Operating Expenses excluding Depreciation" is a direct, first-class configuration, not a workaround. A row also declares its display balance (Debit, Credit, or Net), an indent level, and whether it is a Subtotal row that sums every row above it since the last subtotal. A Calculated row references other rows by formula (e.g. `Gross Profit = Row[Revenue Total] - Row[COGS Total]`) without re-listing accounts.

Column Definitions — each column picks a period basis (Current Period, Year-to-Date, a comparative prior period, or a custom range), a value basis (Debit, Credit, Net Balance, period Movement, or point-in-time Closing Balance — Movement for P&L-style activity reports, Closing Balance for Balance-Sheet/Trial-Balance-style position reports), and an optional dimension filter (e.g. "Branch A only").

Display mode — every report toggles between **Summary** (one figure per defined row) and **Detailed** (each row expands to show its individual constituent accounts, or their transaction lines). Trial Balance and P&L ship as two pre-built, editable Report Definitions using exactly this engine — cloning either into a custom variant (a Branch Contribution Report, a Departmental Overhead Report) requires no code change.

DOCUMENT 3

Entity-Relationship Diagram

The new tables, how they relate to each other, and where they hook into what already exists.

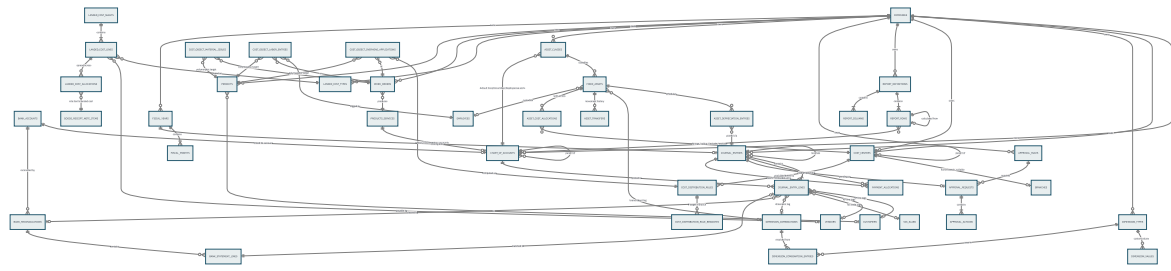


Fig. 3 – Bold entities are new; BRANCHES, CUSTOMERS, VENDORS, TAX_SLABS, BANK_ACCOUNTS, PRODUCTS_SERVICES already exist.

New entities at a glance

ENTITY	PURPOSE	STATUS
chart_of_accounts	Typed, hierarchical account list, per company	New
journal_entries	Balanced entry header — the audit unit	New
journal_entry_lines	One debit or credit per account, per entry	New
payment_allocations	Many-to-many receipt/payment ↔ invoice/bill application	New
dimension_types	Company-configurable dimension definitions	New
dimension_values	Value list for Custom-type dimensions (hierarchical)	New
dimension_combinations	Deduplicated tag-set surrogate ("ledger dimension")	New
dimension_combination_entries	Normalized breakdown of a combination	New
cost_centers	Department/Cost Center master, optional home branch, hierarchical	New
report_definitions	A named, saved report — Trial Balance and P&L ship as two seeded rows here	New
report_rows	One line of a report: range/rollup/list selection + include/exclude + display rules	New
report_columns	One column: period basis, value basis, dimension filter	New
bank_reconciliations	One statement-period reconciliation session per bank account	New

ENTITY	PURPOSE	STATUS
bank_statement_lines	Imported/entered statement lines, matched against journal lines	New
approval_rules	Amount-threshold, multi-level routing per document type	New
approval_requests	One document's approval progress — polymorphic to JE/Bill/Payment/Credit Note	New
approval_actions	Approve/reject history — actor, level, remarks, timestamp	New
asset_classes	Classification with default life/method/accounts, inherited by assets	New
fixed_assets	The asset register — cost, method, status, running NBV	New
asset_depreciation_entries	Per-asset, per-period computed depreciation, linked to its posted entry	New
asset_transfers	From/to branch, cost center, and custodian history per movement	New
asset_cost_allocations	Optional % split of one asset's depreciation across several cost centers	New
landed_cost_types	Company-configured cost categories with a default Capitalize/Expense treatment	New
landed_cost_sheets / lines	Multi-payee import cost sheet attached to a GRN	New
landed_cost_allocations	Per-item share of a capitalized cost line — builds the true landed unit cost	New
cost_distribution_rules	One engine, three targets — pooled cost center distributed to Branches,	New

ENTITY	PURPOSE	STATUS
	Work Orders, <i>or</i> Projects	
work_orders	Cost Object — accumulates to a per-unit manufacturing cost	New
projects	Cost Object — same accumulation, actual-vs-budget instead of a unit cost	New
cost_object_material_issues / labor_entries / overhead_applications	Shared polymorphic tables feeding either a Work Order or a Project	New
fiscal_years	Contains N fiscal_months; the actual Year-End close/lock/rollover happens here	New
fiscal_months	Gains is_adjustment_period — a restricted, final post-close correction window	Modified
stock_ledger_transactions	Gains unitCost / totalCost columns	Modified
products_services / product_categories	glGroupMappings free text replaced by real account FKs	Modified
bank_accounts	Gains a linked GL account	Modified
journal_entry_lines	Gains bank_reconciliation_id / reconciled_at, set only for cleared bank-account lines	Modified
tax_slabs	Gains Output/Input VAT account links	Modified

DOCUMENT 4

Technical Design / Solution Design

Column-level schema, the posting engine's contract, API surface, and the rollout sequence — what an engineer opens first.

4.1 New table schemas

chart_of_accounts

COLUMN	TYPE	NOTES
id	uuid pk	UUIDv7, matches app convention
company_id	uuid fk	→ companies
account_code	text	Unique per company; immutable once posted
account_name	text	Translatable (en/ar/ur)
account_type	text	Asset · Liability · Equity · Revenue · Expense
account_sub_type	text	e.g. CurrentAsset, FixedAsset, Inventory
normal_balance	text	Debit · Credit — derived, not independently editable
parent_account_id	uuid fk, nullable	Self-reference, rollup hierarchy
is_control_account	boolean	Default false
allow_manual_journal	boolean	Forced false when is_control_account
is_active	boolean	Default true; deactivate, never delete

journal_entries

COLUMN	TYPE	NOTES
id	uuid pk	
company_id	uuid fk	
entry_number	text	Via documentNumbering registry key journalEntry
entry_date	text	Validated against fiscalMonths, same as every other document
fiscal_month_id	text fk	Denormalized for fast period queries
source_type	text	Manual · SalesInvoice · PurchaseBill · Payment · Receipt · Allocation · PeriodClose · Payroll...
source_id	uuid, nullable	Polymorphic, same convention as vouchers.referenceId
status	text	Draft · PendingApproval · Posted · Rejected · Reversed
reversal_of_entry_id	uuid fk, nullable	Self-reference
created_by_id / created_at	uuid / timestamp	

journal_entry_lines

COLUMN	TYPE	NOTES
id / journal_entry_id / line_number	uuid / uuid / int	
account_id	uuid fk	→ chart_of_accounts
debit / credit	decimal(14,2)	Exactly one non-zero per line
customer_id / vendor_id / tax_slab_id	uuid, nullable	Fixed, strongly-typed — subledger integrity, not part of the dynamic bag
dimension_combination_id	uuid fk, nullable	→ dimension_combinations — Branch/Department/custom tags resolve through here

dimension_types / dimension_values / dimension_combinations / dimension_combination_entries

TABLE	KEY COLUMNS
dimension_types	company_id · code · name · value_source_type (LinkedEntity Custom) · linked_entity_table · is_hierarchical · is_active
dimension_values	dimension_type_id · code · name · parent_value_id (self-ref, arbitrary depth)
dimension_combinations	company_id · dedup_key (hash of member tags) · cached_display_label
dimension_combination_entries	combination_id · dimension_type_id · entity_id (dimension_values.id, or linked table's row id)
cost_centers	company_id · code · name · home_branch_id (nullable) · parent_cost_center_id (self-ref) · is_active
payment_allocations	company_id · journal_entry_id · allocated_to_type · allocated_to_id · allocated_amount

report_definitions / report_rows / report_columns

TABLE	KEY COLUMNS
<code>report_definitions</code>	<code>company_id</code> · <code>name</code> · <code>type</code> (PnL BalanceSheet TrialBalance Custom) · <code>is_system_seeded</code> · <code>default_display_mode</code> (Summary Detailed) · <code>is_active</code>
<code>report_rows</code>	<code>report_definition_id</code> · <code>row_number</code> · <code>label</code> · <code>selector_type</code> (Range ParentRollup ExplicitList) · <code>range_start/range_end</code> · <code>parent_account_id</code> · <code>include_account_ids[]</code> · <code>exclude_account_ids[]</code> · <code>balance_display</code> (Debit Credit Net) · <code>is_subtotal</code> · <code>indent_level</code> · <code>calc_formula</code> (nullable, refs other <code>row_numbers</code>)
<code>report_columns</code>	<code>report_definition_id</code> · <code>column_number</code> · <code>label</code> · <code>period_basis</code> (CurrentPeriod YTD PriorPeriod CustomRange) · <code>value_basis</code> (Debit Credit Net Movement ClosingBalance) · <code>dimension_filter</code> (jsonb, optional)

Rendering a report: for each row, resolve its account set (base selector, then apply include/exclude on top); for each column, sum matching `journal_entry_lines` by that account set, period, value basis, and dimension filter. Summary mode shows the row total; Detailed mode expands it into its constituent accounts. Trial Balance and P&L are two rows in `report_definitions` with `is_system_seeded = true` — editable and cloneable, not hardcoded functions.

bank_reconciliations / bank_statement_lines

TABLE	KEY COLUMNS
<code>bank_reconciliations</code>	<code>company_id</code> · <code>bank_account_id</code> · <code>statement_date</code> · <code>statement_ending_balance</code> · <code>gl_balance_at_reconciliation</code> · <code>status</code> (InProgress Completed) · <code>completed_by_id</code> · <code>completed_at</code>
<code>bank_statement_lines</code>	<code>bank_reconciliation_id</code> · <code>statement_date</code> · <code>description</code> · <code>amount</code> · <code>matched_journal_entry_line_id</code> (nullable) · <code>is_matched</code>

approval_rules / approval_requests / approval_actions

TABLE	KEY COLUMNS
<code>approval_rules</code>	<code>company_id</code> · <code>applies_to_type</code> (<code>JournalEntry</code> <code>VendorBill</code> <code>VendorPayment</code> <code>CreditNote</code>) · <code>level</code> · <code>min_amount</code> · <code>max_amount</code> (nullable) · <code>approver_role_id</code> · <code>is_active</code>
<code>approval_requests</code>	<code>company_id</code> · <code>entity_type</code> · <code>entity_id</code> (polymorphic) · <code>requested_by_id</code> · <code>requested_at</code> · <code>current_level</code> · <code>status</code> (<code>Pending</code> <code>Approved</code> <code>Rejected</code>)
<code>approval_actions</code>	<code>approval_request_id</code> · <code>level</code> · <code>actor_id</code> · <code>action</code> (<code>Approved</code> <code>Rejected</code>) · <code>acted_at</code> · <code>remarks</code>

fiscal_years (and `fiscal_months.is_adjustment_period`)

TABLE	KEY COLUMNS
<code>fiscal_years</code>	<code>company_id</code> · <code>year_label</code> · <code>start_date</code> · <code>end_date</code> · <code>status</code> (<code>Open</code> <code>Closing</code> <code>Closed</code>) · <code>closing_journal_entry_id</code> · <code>closed_at</code> · <code>closed_by_id</code>

No separate "readiness check" table — GET `/api/gl/fiscal-years/:id/readiness` computes the six-point checklist live against `fiscal_months`, `bank_reconciliations`, `asset_depreciation_entries`, `tax_returns`, `cost_distribution_runs`, and every transactional table's status column — it's a query, not stored state that could drift from reality. The Adjustment Period is just a `fiscal_months` row flagged `is_adjustment_period = true`, gated by its own permission, not a new mechanism.

asset_classes / fixed_assets / asset_depreciation_entries

TABLE	KEY COLUMNS
asset_classes	company_id · name · default_useful_life_months · d (StraightLine DecliningBalance UnitsOfProduction) default_cost_account_id · default_accum_dep_accoun default_dep_expense_account_id
fixed_assets	company_id · asset_code · name · asset_class_id · acquisition_date · acquisition_cost · salvage_valu useful_life_months · method · rate (nullable, decl balance) · in_service_date (nullable) · status (UnderConstruction InService FullyDepreciated Reti · cost_account_id · accum_dep_account_id · dep_expense_account_id · dimension_combination_id custodian_employee_id (nullable) · current_accumulated_depreciation · current_net_bo disposal_date · disposal_proceeds
asset_depreciation_entries	fixed_asset_id · fiscal_month_id · depreciation_am accumulated_depreciation_after · net_book_value_af journal_entry_id · unique(fixed_asset_id, fiscal_m the idempotency guard
asset_transfers	fixed_asset_id · transfer_date · from_dimension_co · to_dimension_combination_id · from_custodian_id to_custodian_id · reason · requested_by_id · appro (nullable)
asset_cost_allocations	fixed_asset_id · cost_center_id · allocation_perce asset must equal 100

A depreciation run groups its posting by (Depreciation Expense account × dimension combination), not one GL line per asset — hundreds of assets still produce a compact, legible journal entry. Per-asset detail lives in `asset_depreciation_entries`, which is what the Fixed Asset Register report and any drill-down actually reads.

**landed_cost_types / landed_cost_sheets /
landed_cost_lines / landed_cost_allocations**

TABLE	KEY COLUMNS
<code>landed_cost_types</code>	<code>company_id</code> · <code>name</code> (Freight, Duty, Insurance, Clearing Agent Commission, Bank Charges...) · <code>default_treatment</code> (Capitalize ExpenseDirect) · <code>default_expense_account_id</code> (used when ExpenseDirect)
<code>landed_cost_sheets</code>	<code>company_id</code> · <code>grn_id</code> · <code>sheet_date</code> · <code>status</code> (Draft Posted)
<code>landed_cost_lines</code>	<code>landed_cost_sheet_id</code> · <code>landed_cost_type_id</code> · <code>payee_type</code> (Vendor GovernmentAuthority DirectBank) · <code>vendor_id</code> (nullable) · <code>amount</code> · <code>treatment</code> (inherits type default, overridable) · <code>allocation_basis</code> (ByValue ByQuantity Manual) · <code>accrual_account_id</code>
<code>landed_cost_allocations</code>	<code>landed_cost_line_id</code> · <code>goods_receipt_note_item_id</code> · <code>allocated_amount</code>

Each line's own `vendor_id` is what lets one shipment generate several independent AP liabilities — the goods vendor, the freight forwarder, the customs agent, and the insurer are each their own payee, each individually visible in AP Aging, none of them forced onto the primary Purchase Bill. `landed_cost_lines` is a plain one-to-many child of `landed_cost_sheets` — nothing in the schema caps or presumes the count; a domestic purchase with only a delivery charge is one line, a complex import might be six or seven, and the posting/allocation logic is identical either way.

cost_distribution_rules — the unified allocation engine

TABLE	KEY COLUMNS
<code>cost_distribution_rules</code>	<code>company_id</code> · <code>source_cost_center_id</code> (the pool) (<code>Branch</code> <code>WorkOrder</code> <code>Project</code>) · <code>basis_type</code> (<code>FixedPercent</code> <code>RevenuePercent</code> <code>PerDirectLaborHo</code>) · <code>rate</code> (for the per-activity basis types) · <code>dis</code> <code>is_active</code>
<code>cost_distribution_rule_branches</code>	<code>rule_id</code> · <code>branch_id</code> · <code>weight_percent</code> — only p = <code>Branch</code>
<code>cost_distribution_runs</code>	<code>company_id</code> · <code>rule_id</code> · <code>fiscal_month_id</code> · <code>stat</code> <code>journal_entry_id</code>

One table drives branch reclass, manufacturing absorption, and project overhead alike — only `target_type` and `basis_type` change. The same "Configure Allocation Rule" screen and the same preview-then-post run serve all three; there is no separate UI or engine per scenario.

work_orders / projects — Cost Objects, and their shared cost-accumulation tables

TABLE	KEY COLUMNS
<code>work_orders</code>	<code>company_id</code> · <code>work_order_number</code> · <code>product_id</code> · <code>quantity_planned</code> · <code>quantity_completed</code> · <code>status</code> · <code>dimension_combination_id</code> · <code>accumulated_material_cost</code> · <code>accumulated_labor_cost</code> · <code>accumulated_overhead_cost</code> · <code>total_cost</code> · <code>unit_cost</code>
<code>projects</code>	<code>company_id</code> · <code>project_number</code> · <code>name</code> · <code>customer_id</code> (nullable) · <code>budgeted_cost</code> · <code>status</code> · <code>dimension_combination_id</code> · <code>accumulated_material_cost</code> · <code>accumulated_labor_cost</code> · <code>accumulated_overhead_cost</code> · <code>total_actual_cost</code>
<code>cost_object_material_issues</code>	<code>cost_object_type</code> (WorkOrder Project) · <code>cost_object_id</code> · <code>product_id</code> · <code>quantity</code> · <code>unit_cost</code> · <code>total_cost</code> · <code>journal_entry_id</code>
<code>cost_object_labor_entries</code>	<code>cost_object_type</code> · <code>cost_object_id</code> · <code>employee_id</code> · <code>hours_logged</code> · <code>labor_rate</code> · <code>labor_cost</code> · <code>journal_entry_id</code>
<code>cost_object_overhead_applications</code>	<code>cost_object_type</code> · <code>cost_object_id</code> · <code>cost_distribution_rule_id</code> · <code>basis_quantity</code> · <code>applied_overhead_cost</code> · <code>journal_entry_id</code>

Three tables, not six — a Work Order and a Project both post through the same polymorphic `cost_object_*` tables rather than each getting their own bespoke material/labor/overhead schema. Unit cost, where it applies: $\text{total_cost} \div \text{quantity_completed}$.

4.2 Posting engine contract

POSTJOURNALENTRY(TX, PARAMS)

Takes a company id, source type/id, entry date, description, and an array of lines (account, debit/credit, optional customer/vendor/tax/dimension tags). Inside the caller's existing `db.transaction()`: validates the entry date against the open fiscal month (reusing `validateTransactionDate`), resolves each line's dimension tags to a `dimension_combination_id` (find-or-create against the dedup key), asserts $\Sigma\text{debit} = \Sigma\text{credit}$, rejects any line whose account is a control account unless the caller is itself a subledger-internal call, checks `approval_rules` for a match on entity type + amount — no match posts immediately as today; a match instead creates an `approval_requests` row at level 1 and returns with `status='PendingApproval'`, writing nothing to the ledger yet. The actual account-balance-affecting insert only happens when `approveApprovalRequest()` clears the final required level; a reject never touches `journal_entry_lines` at all. Nothing outside this pair of functions ever writes to `journal_entry_lines` directly.

4.3 API surface (new routes)

METHOD & PATH	PURPOSE
GET/POST /api/gl/accounts	Chart of Accounts CRUD (create/update = draft, deactivate = its own action)
GET /api/gl/accounts/:id/references	The "Linked Configuration" tab query — what points at this account
POST /api/gl/journal-entries	Manual JE — draft then a separate /post action
POST /api/gl/journal-entries/:id/reverse	Posts the reversing entry, links it back
GET /api/gl/ledger? accountId=&from=&to=&dimensions=	General Ledger Detail — the drill-down every report ultimately uses
POST /api/gl/dimension-types	Configure a new dimension type (Custom or LinkedEntity)
POST /api/gl/cost-centers/:id/allocation-rule	Configure basis + destination weights
POST /api/gl/allocations/run	Executes the month-end allocation, posts the reclass entry
POST /api/transactions/months (extended)	Close action now locks only — no P&L zeroing at the monthly level
GET /api/gl/fiscal-years/:id/readiness	Live six-point checklist; Year-End Close is blocked until every item is true
POST /api/gl/fiscal-years/:id/close	Posts the one annual closing entry, locks the year, opens the new year's first month
POST /api/gl/fiscal-years/:id/adjustment-period/open	Opens the restricted post-close correction window
POST/GET /api/gl/report-definitions	CRUD for report layouts; clone a seeded definition into a custom one
GET /api/gl/report-definitions/:id/run	Evaluates rows × columns against journal_entry_lines for a date range/dimension scope

METHOD & PATH	PURPOSE
POST /api/gl/bank-reconciliations	Opens a session for a bank account + statement period
POST /api/gl/bank-reconciliations/:id/match	Links a statement line to a journal line (or auto-match pass)
POST /api/gl/bank-reconciliations/:id/complete	Blocked unless the reconciling formula balances to zero
POST/GET /api/gl/approval-rules	Configure thresholds and routing per document type
POST /api/gl/approval-requests/:id/approve	Rejects if actor = requester; advances level or triggers the real post
POST /api/gl/approval-requests/:id/reject	Returns the source document to Draft with remarks
POST/GET /api/gl/fixed-assets	Register an asset; class defaults pre-fill, all overridable
POST /api/gl/fixed-assets/:id/put-in-service	CWIP → real Cost account, starts the depreciation clock
GET /api/gl/depreciation-runs/preview?period=	Computed amounts per asset, before anything posts
POST /api/gl/depreciation-runs	Posts the grouped entry; blocked if the period already ran
POST /api/gl/fixed-assets/:id/dispose	Clears Cost + Accum. Depreciation, posts Gain/Loss on Disposal (proceeds > 0)
POST /api/gl/fixed-assets/:id/retire	Same mechanics as dispose, proceeds fixed at 0, status = Retired
POST /api/gl/fixed-assets/:id/transfer	New branch/cost-center/custodian, applies from the transfer date forward
PUT /api/gl/fixed-assets/:id/cost-allocation	Set the % split across cost centers; rejected unless it sums to 100

METHOD & PATH	PURPOSE
GET /api/gl/fixed-assets/:id/transfer-note GET /api/gl/fixed-assets/:id/disposal-note	Print-ready form, same rendering path as every other movement document
POST/GET /api/gl/landed-cost-types	Configure cost categories and their default treatment
POST /api/inventory/grn/:id/landed-costs	Add cost lines to a posted GRN, preview allocated per-item landed cost
POST /api/inventory/grn/:id/landed-costs/post	Posts the capitalize/expense entries; item averageCost recomputed from landed cost
POST/GET /api/gl/cost-distribution-rules	One config screen and API for Branch, Work Order, <i>and</i> Project targets
GET /api/gl/cost-distribution-rules/:id/preview?period=	Computed split before anything posts — same preview used for every target type
POST /api/gl/cost-distribution-rules/:id/run	Posts the distribution entry; blocked if already run for the period
POST/GET /api/gl/work-orders	Create/manage; material issue, labor entry, and completion are its own sub-actions
POST/GET /api/gl/projects	Same cost-accumulation shape as Work Orders, actual-vs-budget instead of unit cost
POST /api/gl/{work-orders projects}/:id/complete	Crystallizes total cost; Work Orders additionally compute a unit cost into Finished Goods

4.4 Numbering & permissions

documentNumbering.ts's DOCUMENT_TYPE_REGISTRY gains one entry — journalEntry, prefix JV — following the existing one-line-addition pattern; no other file needs to change for numbering to work.

The existing permissionSchema.ts shape — one PERMISSION_MODULES entry per feature, boolean leaves, roles OR-merged, a big irreversible action getting its own leaf (already the convention behind fiscalMonths.close, taxReturns.file,

`banks.transfer`) — needs no structural change to cover this design. It extends by adding modules, the same way it always has:

MODULE	LEAVES	WHY
chartOfAccounts	create · read · update · deactivate	No hard delete once posted
journalVoucher	create · read · update · submit · approve · post · reverse	post/reverse/approve are the irreversible-action leaves
purchaseBills existing module	create · read · update · delete (<i>unchanged</i>) + approve (<i>new</i>)	FR-APR-01 names Vendor Bill as approval-eligible; base CRUD already exists today, only the approval leaf is new
vendorPayments new module	create · read · approve	"create" here means apply/settle a payment across one or more bills — a genuinely new capability, deliberately not folded into purchaseBills.update. Money leaving is exactly why this gets its own approve leaf.
invoice existing module	create · read · update · delete (<i>unchanged</i>)	No new approve leaf — issuing an invoice isn't a cash-outflow event; deliberately not gated by the approval cycle
customerReceipts new module	create · read	The AR mirror of vendorPayments — apply/settle a receipt across one or more invoices. No approve leaf: receiving money is the one flow the approval cycle deliberately doesn't gate (see the asymmetry note below)
creditNote / debitNote existing module	create · read · update · delete (<i>unchanged</i>) + approve (<i>new</i>)	FR-APR-01 names Credit Note specifically — it reverses recognized revenue, a real control point
dimensionTypes	create · read · update	Configuring Branch/Department/Project-style dimensions

MODULE	LEAVES	WHY
costCenters	create · read · update	
costDistributionRules	create · read · update · run	run posts real GL entries
reportDefinitions	design · read	New split — designing a layout is a different privilege than viewing a seeded report's output, still gated by the existing per-report reports.* leaves
bankReconciliation	create · read · match · complete · reopen	reopen on a completed session is the elevated action, per FR-BANK-05
approvalRules	create · read · update	Sensitive by nature — this role controls who can weaken financial controls
fixedAssets	create · read · update · transfer · dispose	Disposal/retirement is irreversible
assetClasses	create · read · update	
depreciationRun	preview · post	
landedCost	create · read · post	
workOrders / projects	create · read · update · complete	Completion crystallizes cost — Work Order into Finished Goods, Project into final actuals
fiscalYears	read · close · openAdjustmentPeriod	Two of the most consequential actions in the whole system — kept as separate, rare leaves, not folded into fiscalMonths.close

DELIBERATE ASYMMETRY: OUTFLOWS GATED, INFLOWS NOT

Vendor Bills and Vendor Payments both carry an approve leaf; Sales Invoices and Customer Receipts carry neither. This isn't an oversight — approval controls in every real AP/AR system concentrate on money *leaving* the company, because that's the actual fraud/error surface. Recording that a customer paid you is not a control point the way authorizing a payment to a vendor is. Worth stating outright rather than leaving it to look like an inconsistency.

DELIBERATELY NOT A PERMISSION

Amount-based approval thresholds ("this role can approve up to 50,000 SAR without a second sign-off") are not expressed as leaves — they live in `approval_rules`, keyed on amount brackets, because permissions answer "can this role act at all," not "does this specific transaction's size require another sign-off." Likewise, segregation of duties — one person can't approve their own submission — is a runtime check (`approverId ≠ requestedById`) at the moment of approval, layered on top of the `journalVoucher`.approve leaf, not expressed by it. And branch-scoped visibility for a Branch Manager's P&L view reuses the existing `userBranches/allowedBranchIds` mechanism already in the app — no schema change needed there at all.

4.5 Rollout sequence

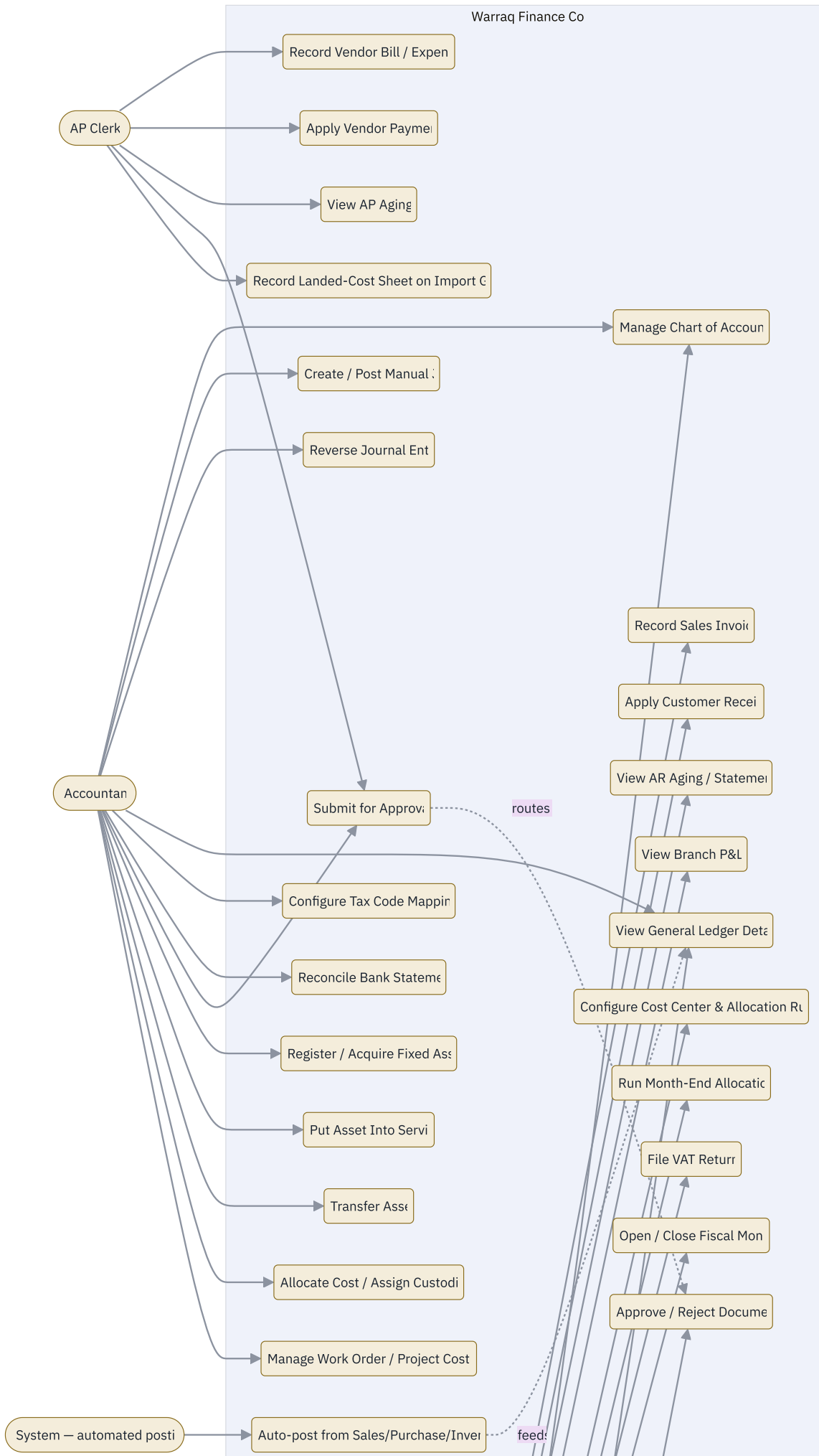
#	STEP	RISK
1	Schema only — every new table added, nullable/additive, nothing reads or writes yet	Zero
2	Seed Chart of Accounts — new companies via provisioning, existing via a one-off backfill script	Low
3	Dual-post — every existing route additionally posts a JE alongside its untouched legacy fields	Medium — verification window
4	Cut reports over one at a time, Trial Balance first, old and new compared side by side	Medium
5	Cut period close over to the server-verified mechanism	Medium
6	Historical backfill — retroactive entries for existing documents, dry-run first, explicit sign-off before a real run	Highest — real financial history

Full step-by-step detail lives in the working plan file already agreed this session; this table is the summary view.

DOCUMENT 5

Use Case Diagram

Who does what to this system — grouped by area, one system boundary.



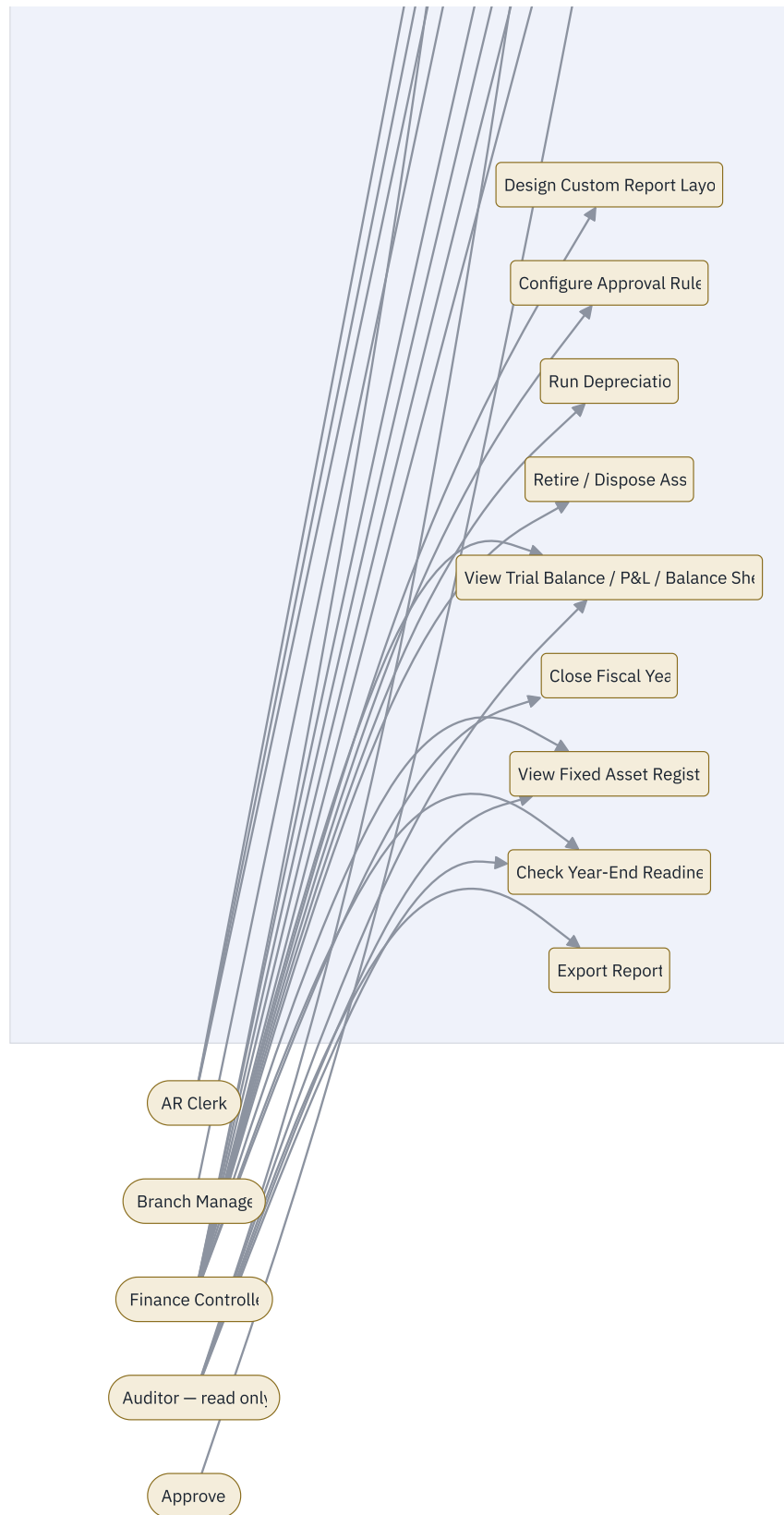


Fig. 4 – Eight actors, thirty-five use cases, one system boundary. UC11 (Configure Cost Center & Allocation Rule) already covers Branch, Work Order, and Project targets – one screen, not three.

DOCUMENT 6

UX — Use Case Detail

Every use case above, expanded: who triggers it, what has to be true first, exactly what's on screen, and what it leaves behind. Four of the more involved journeys get their own flow diagram.

6.1 Full use case specification

USE CASE	ACTOR	PRECONDITIONS	ON-SCREEN OPTIONS
Manage Chart of Accounts	Accountant Controller	chartOfAccounts.create/update permission	New Account, Edit t (General/Posting Rules/Default Dimensions/Linked Config/Balance), Deactivate
Create / Post Manual JE	Accountant	journalVoucher.create; open fiscal month	Add line, pick account (non-control only), debit or credit, tag dimensions, running balance indicator, S Draft, Post
Reverse Journal Entry	Accountant	journalVoucher.reverse; entry is Posted, not already reversed	View entry, Reverse button, reason note
View General Ledger Detail	Accountant Auditor	reports.generalLedgerDetail	Filter by account, date range, branch, department; drill from report total into this
Record Vendor Bill / Expense	AP Clerk	purchaseBills.create or expenses.create; open month	Link to GRN (or non item lines with map accounts, tax code,
Apply Vendor Payment	AP Clerk	Open payable bills exist	Pick bank account, one or many open bills allocate amount per Pay
View AP Aging	AP Clerk Controller	reports.outstanding	Filter by vendor, aging buckets, export
Record Sales Invoice	AR Clerk	invoice.create; open month	Customer picker, item tax, discount; existing screen, now GL-integrated underneath
Apply Customer Receipt	AR Clerk	Open receivable invoices exist	Pick bank/cash account select one or many invoices, allocate, o

USE CASE	ACTOR	PRECONDITIONS	ON-SCREEN OPTIONS
			unapplied on-account Receive
View AR Aging / Statement	AR Clerk Controller	reports.outstanding	Filter by customer, buckets, print state
Configure Cost Center & Allocation Rule	Controller	chartOfAccounts config permission	New Cost Center, select home Branch (or leave blank for shared), if shared: Basis (Fixed%/Revenue% destination weights
Run Month-End Allocation	Controller	Open fiscal month; allocation rules configured	Preview computed before posting, Run
View Branch P&L	Branch Manager Controller	reports scoped to own branch (Branch Manager) or all (Controller)	Branch selector, Contribution Margin fully-loaded Net Profit toggle
Configure Tax Code Mapping	Accountant	taxSlabs config permission	Link each tax slab to Output/Input VAT a
File VAT Return	Controller	taxReturns.file; quarter Generated, not yet Filed	Review computed file File (irreversible)
Open / Close Fiscal Month	Controller	fiscalMonths.open / .close	Open next month; Close oldest open month (server recomputes verifies)
View Trial Balance / P&L / Balance Sheet	Controller Auditor	Per-report leaf permission	Date range, branch/department drill to GL Detail
Export Reports	Auditor Controller	Report view permission	PDF / Excel / CSV

USE CASE	ACTOR	PRECONDITIONS	ON-SCREEN OPTIONS
Auto-post from Sales/Purchase/Inventory	System	n/a — fires inside each source route's own transaction	No UI — internal call postJournalEntry()
Design Custom Report Layout	Controller	Report-definition permission	Clone Trial Balance, or start blank; add r (Range/Rollup/List Include/Exclude, Subtotal, Calculated columns (period + v basis + dimension f preview, Save, Summary/Detailed
Reconcile Bank Statement	Accountant	Bank account has posted, uncleared lines	Enter/import staten lines, match against uncleared GL lines (match proposed, m override), post adjustment JE for tr discrepancies, live reconciling-formula indicator, Complete
Configure Approval Rules	Controller	Approval-rule config permission	Pick document type amount bracket, as approver role per le add another level, S
Submit for Approval	Accountant AP Clerk	Document amount matches an active rule	Save as Draft, Subm system shows which rule/level applies be sending
Approve / Reject Document	Approver	A request is pending at the approver's level; approver ≠ submitter	Review lines/amount/dime Approve or Reject w remarks
Register / Acquire Fixed Asset	Accountant	Asset Class exists	Pick class (pre-fills life/method/accoun

USE CASE	ACTOR	PRECONDITIONS	ON-SCREEN OPTIONS
			cost, acquisition date, Service now or CWI
Put Asset Into Service	Accountant	Asset status = UnderConstruction	Confirm in-service c
Run Depreciation	Controller	Open period; not already run for it	Preview computed amount per asset, F
Transfer Asset	Accountant	Asset is InService	New branch/cost-center/custodian, re transfer date
Allocate Cost / Assign Custodian	Accountant	Asset is InService	Set custodian empl optionally split depreciation % across multiple cost center (must sum to 100)
Retire / Dispose Asset	Controller	Asset is InService	Disposal date, proc (0 for Retire), reason
View Fixed Asset Register	Controller Auditor	reports.fixedAssetRegister	Filter by class, branch status; export
Record Landed-Cost Sheet on Import GRN	AP Clerk	GRN posted for an import shipment	Add Line / Remove unlimited count — p type, payee, amount line, choose allocation basis, preview per-unit landed unit cost, Po

USE CASE	ACTOR	PRECONDITIONS	ON-SCREEN OPTIONS
Manage Work Order / Project Costing	Accountant	Work Order or Project open	Issue material, log l (employee + hours) overhead (via the sh Cost Distribution Ru view running Total v Budget, Complete
Check Year-End Readiness	Controller Auditor	Fiscal year exists	Live six-point check (months, bank recs, depreciation, VAT, allocations, no dang drafts) — each item to what's blocking i
Close Fiscal Year	Controller	Readiness checklist fully passed	Review computed f year net income, Clo (irreversible)

6.2 Key flows

Post a manual journal entry

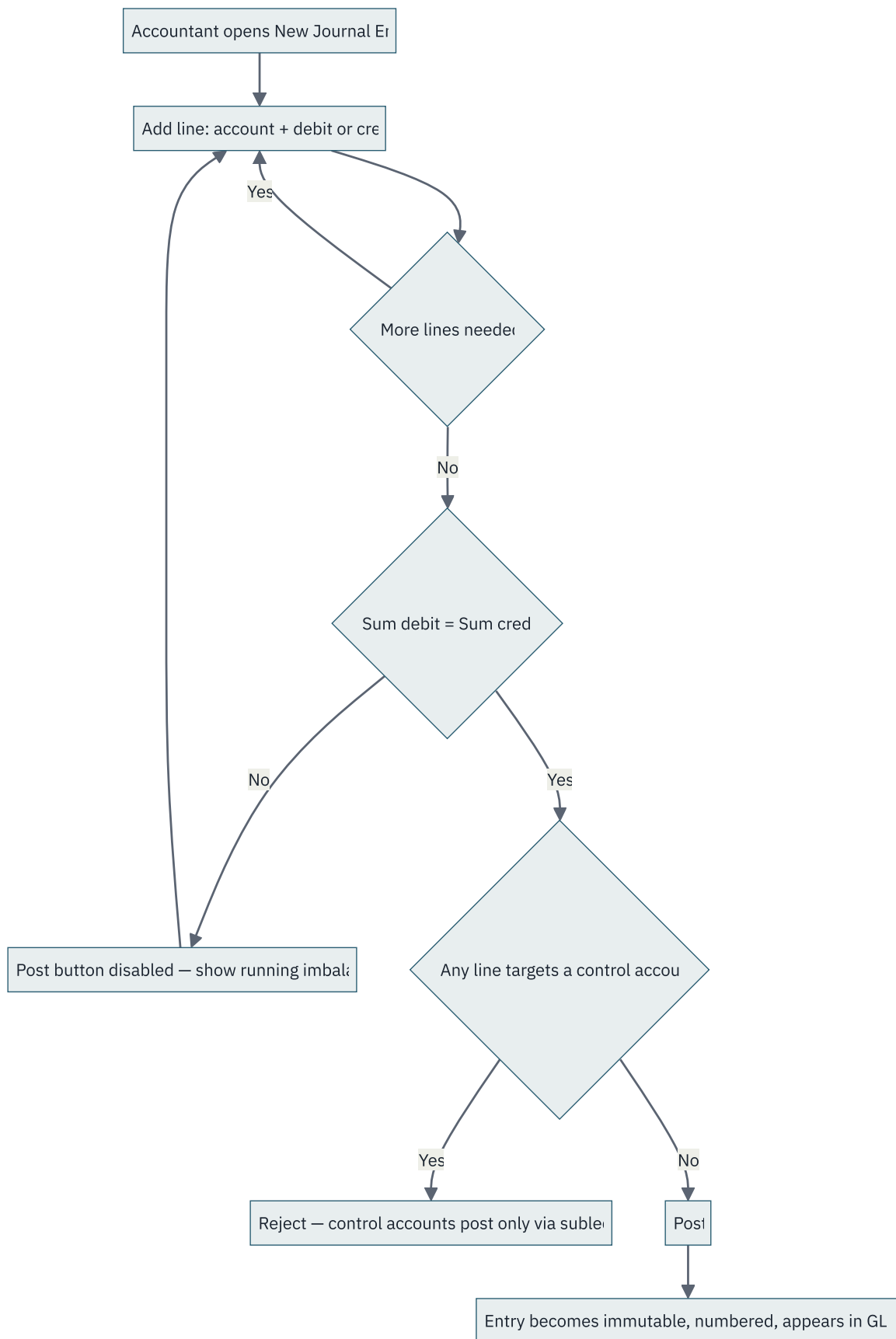


Fig. 5

Record & pay a vendor bill (3-way match)

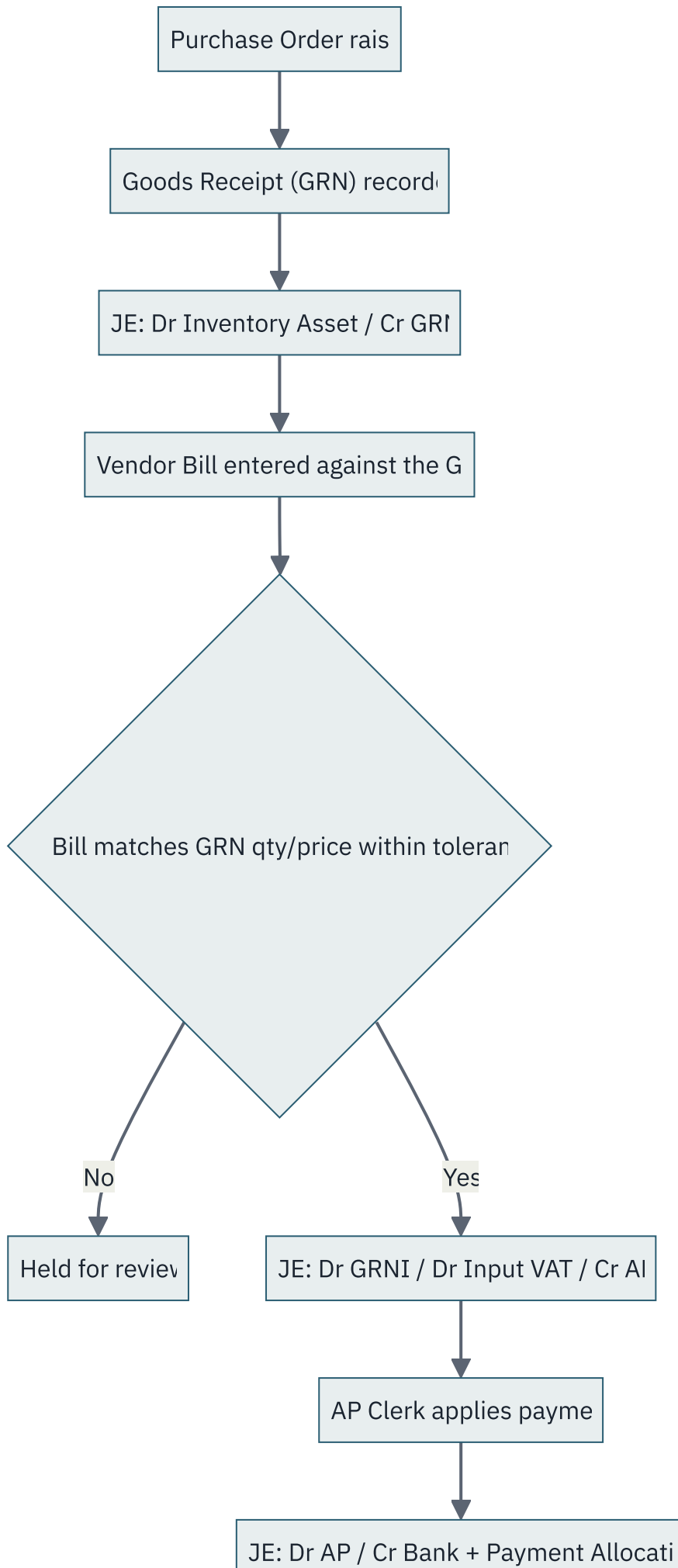


Fig. 6

Apply a customer receipt across multiple invoices

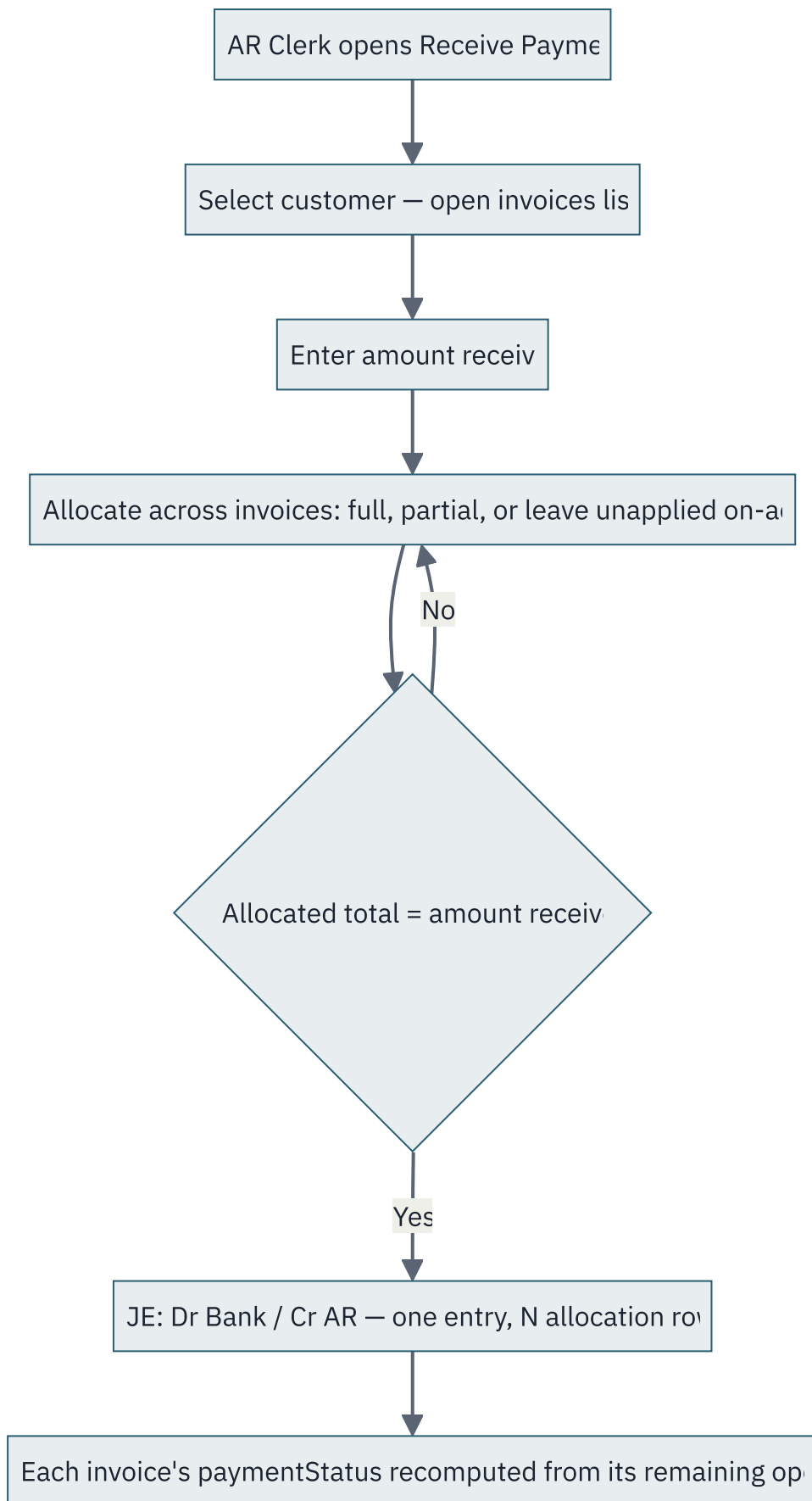


Fig. 7

Run month-end close

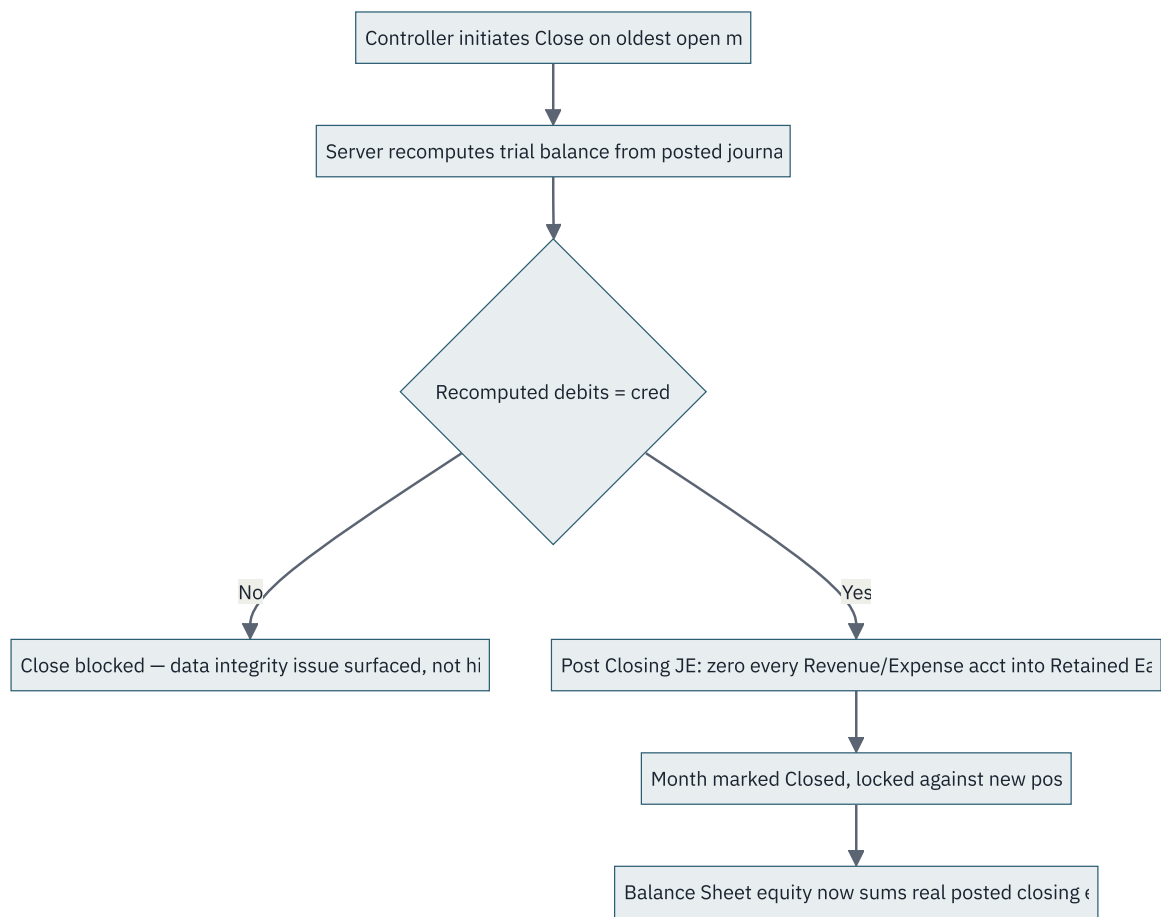


Fig. 8

Reconcile a bank statement

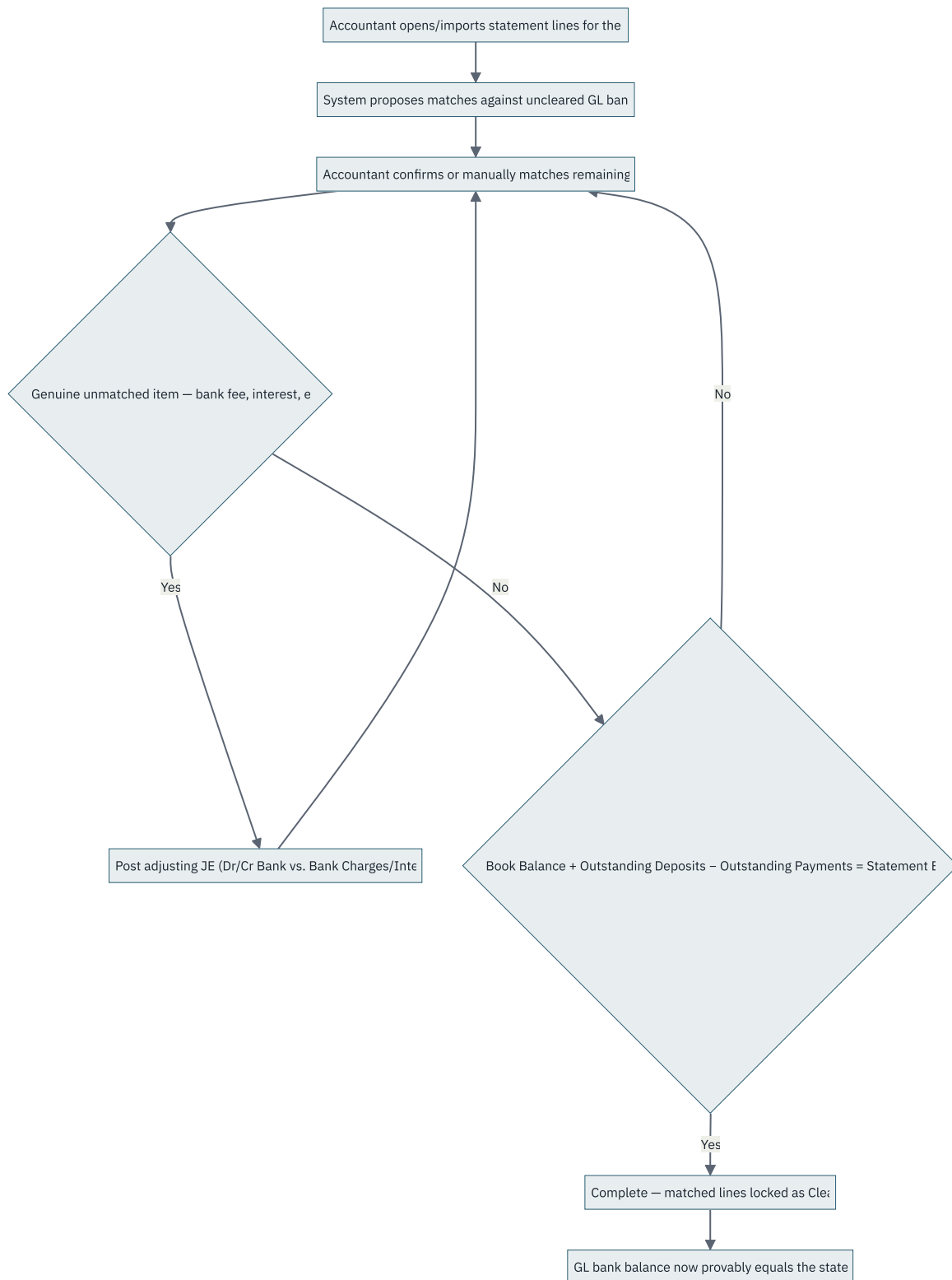


Fig. 9

Submit and approve a high-value vendor payment

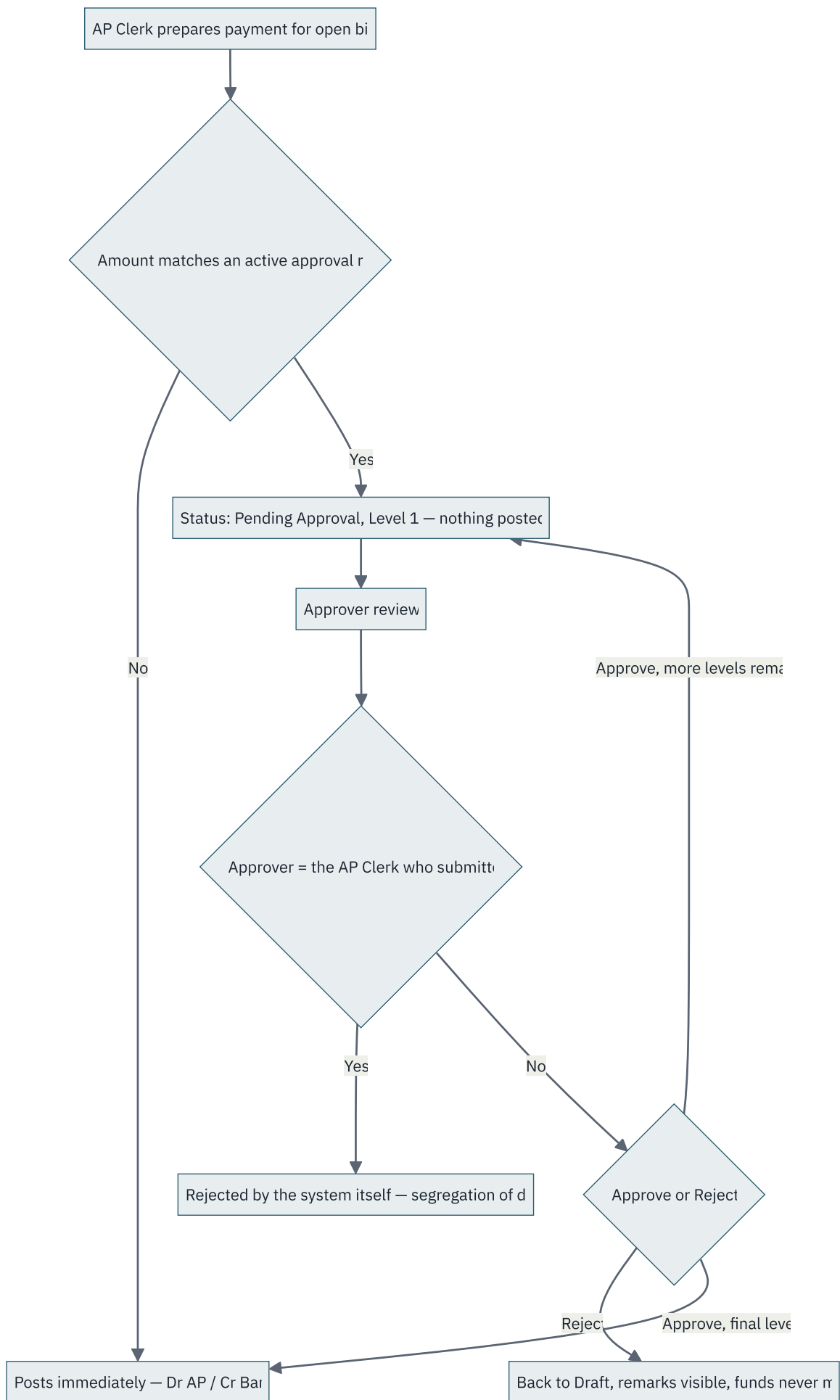


Fig. 10

Run monthly depreciation

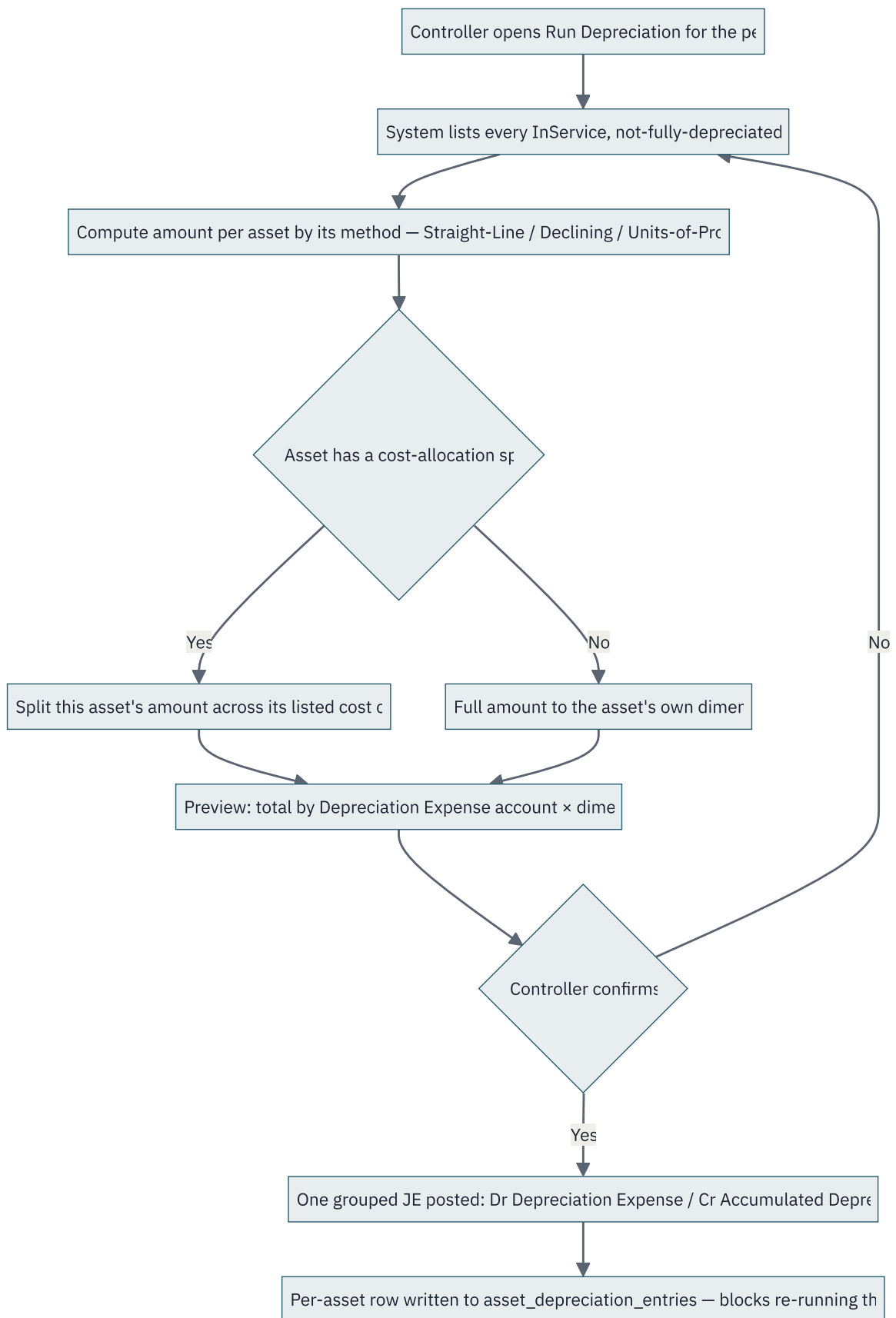


Fig. 11

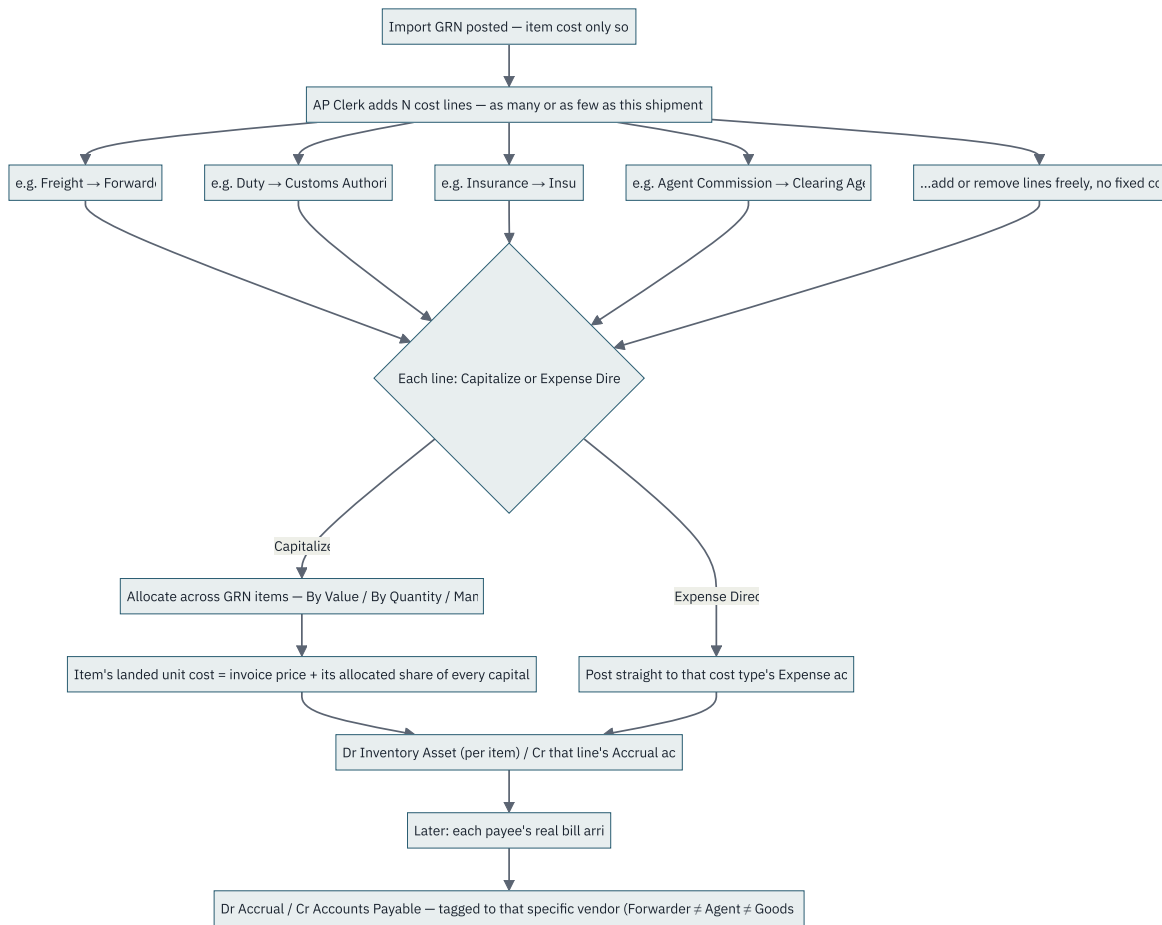


Fig. 12 — Any number of payees, that many AP liabilities, one shipment — two lines or ten, the mechanism doesn't change.

Close a fiscal year

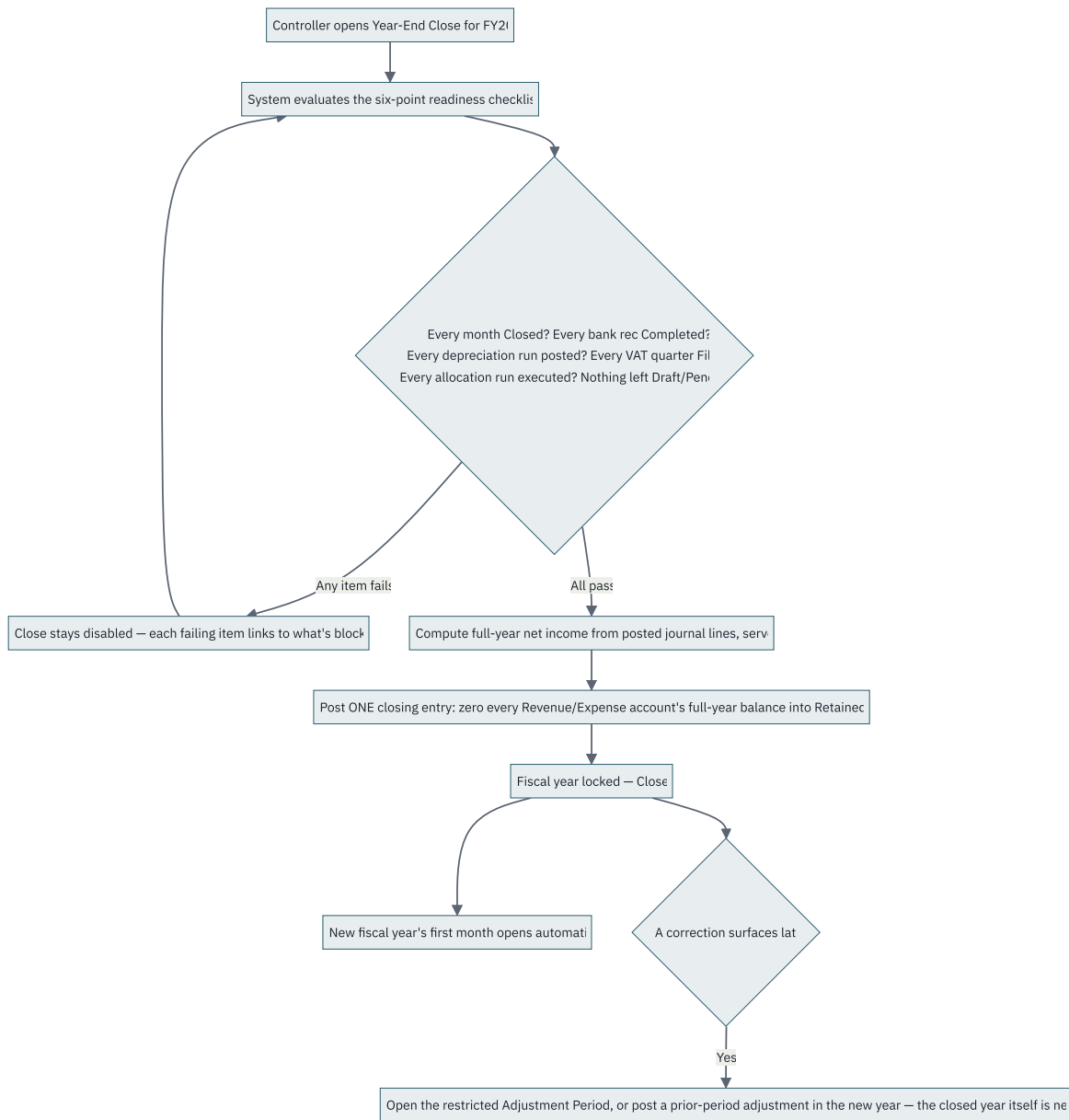


Fig. 13 – Balance Sheet accounts need no carry-forward step at all; only Revenue/Expense (zeroed) and Retained Earnings (incremented) are touched.

REFERENCE

Glossary

Control account	A GL account only a subledger (AR/AP/Inventory/Tax) may post to — never a manual journal.
Dimension	A tag on a journal line describing context beyond the account — Branch, Department, Customer, Vendor, Tax Code.
Dimension combination	A deduplicated, reusable surrogate representing one specific set of dimension tags — keeps dynamic dimensions fast.
GRNI	Goods Received Not Invoiced — clearing account bridging a Goods Receipt and its later matched Bill.
Reclass allocation	Moving a shared cost from a central pool into branch dimensions via a dedicated account, net zero company-wide.
Settlement	Applying a payment/receipt against one or more open invoices/bills — what "payment allocation" implements.
Subledger	A filtered view over journal lines (e.g. all AR lines) — not a separately maintained balance.
Report Definition	A saved combination of Row and Column Definitions that produces a report — Trial Balance and P&L are two seeded examples, not special cases.
Row / Column Definition	A row selects an account set (range, rollup, or list) with include/exclude modifiers and a display rule; a column selects a period and value basis. Every report is rows × columns evaluated against journal lines.
Cleared / Outstanding	A bank-account journal line is Cleared once matched to a statement line during reconciliation; until then it's Outstanding (e.g. a check issued but not yet cashed).
Approval Rule	A configured amount bracket, per document type, that routes a submission through one or more approval levels before it can post.
Segregation of Duties	The person who submits a document is never allowed to also approve it, at any level — enforced server-side.
CWIP	Capital Work-in-Progress — an asset's cost while under construction/fit-out, not yet depreciating; moves to its real Cost account once put In-Service.
NBV	Net Book Value — an asset's Cost minus Accumulated Depreciation at a point in time.

Custodian	The employee currently responsible for a fixed asset — distinct from which branch/cost-center absorbs its depreciation.
Landed Cost	An item's true cost once every cost to get it into stock is included — invoice price plus its allocated share of freight, duty, insurance, and agent fees — not the vendor invoice price alone.
Cost Object	Anything that sums Direct + Allocated cost into a Total — a Branch, a Work Order, a Project, or an inventory item at receipt.
Cost Distribution Rule	One configured rule — pool, basis, destination — that drives branch overhead reclass, manufacturing/project overhead absorption, and (conceptually) landed-cost allocation alike.
Absorption Rate	A predetermined rate (e.g. SAR per direct-labor-hour) used to apply pooled overhead to Work Orders/Projects, set from recent actual or budgeted pool cost ÷ activity level.
Monthly Lock vs. Year-End Close	Locking a month blocks further postings to it; it does not zero any account. Only Fiscal Year-End Close posts the one entry that zeros Revenue/Expense into Retained Earnings.
Adjustment Period	A restricted, final window after a closed year, for genuine audit corrections — used instead of ever reopening the year's real operating months.

REFERENCE

Open decisions carried into this package

- Payroll penalty-deduction posting: net against Salary Expense, or gross salary plus a separate Penalty Recovery account — a reporting-preference choice, not yet made.
- Whether branches itself needs its own hierarchy (Region → City → Branch) — same cheap self-FK pattern as Cost Center if it's ever wanted, not needed for this design specifically.
- Allocation bases beyond Fixed % and Revenue % (headcount, floor area, transaction volume) — deferred until a real case for them shows up.

Companion: full architecture discussion & sample Chart of Accounts, this session.